

Architecture des Ordinateurs — Chapitres 2 à 5

Nom : BARETS

Prénom : Emma

ID Étudiant : 100672190DG

Table des matières

1	Chapitre 2 — La Représentation des Nombres Entiers Positifs	3
1.1	Exercice 2.2 (Bonus)	3
1.2	Exercice 2.3 (A Rendre)	15
1.3	Exercice 2.12 (A Rendre)	23
1.4	Conclusion du chapitre 2	24
2	Sources du chapitre 2	25
	Chapitre 3 — Opérations en Binaire & Représentation des Nombres Signés	26
2.1	Exercice 3.6 (A Rendre)	26
2.2	Exercice 3.14 (A Rendre)	29
2.3	Exercice 3.15 (Bonus)	34
2.4	Conclusion du chapitre 3	36
3	Sources du chapitre 3	37
	Chapitre 4 — Représentation des Nombres Flottants et des Caractères	38
3.1	Exercice 4.3 (A Rendre)	38
3.2	Exercice 4.6 (Bonus)	40
3.3	Exercice 4.13 (A Rendre)	43
3.4	Exercice 4.17 (Bonus)	49
3.5	Conclusion du chapitre 4	52
4	Sources du chapitre 4	54
	Chapitre 5 - Algèbre de Boole	55
4.1	Exercice 5.9 (Bonus)	55
4.2	Exercice 5.10 (À rendre)	56
4.3	Exercice 5.11 (À rendre)	57
4.4	Conclusion du chapitre 5	59
5	Sources du chapitre 5	60

1 Chapitre 2 — La Représentation des Nombres Entiers Positifs

Exercices

Pour ce chapitre 2, vous devez rendre les exercices **2.3** et **2.12**

L'exercice **2.2**, qui vous demandera un peu de programmation, est pour un **Bonus**.

1.1 Exercice 2.2 (Bonus)

Décrire dans le détail, avec des phrases, la suite d'opérations qu'il faut effectuer pour additionner deux nombres écrits en chiffres romains. Écrire ensuite le programme correspondant dans le langage de votre choix (cf. Petit Guide).

Pour vous aider : Une manière "intelligemment-paresseuse" de procéder est d'effectuer l'addition en base dix des nombres saisis par l'utilisateur puis de convertir le résultat à l'aide du programme réalisé dans l'exercice 2.3.

Pour additionner deux nombres écrits en chiffres romains il faut d'abord demander à l'utilisateur d'entrer deux nombres en chiffres romains, par exemple X et V. Ensuite on va convertir chaque nombre romain en nombre entier à l'aide d'une fonction qui va associer chaque symbole romain à sa valeur et applique les règles de soustraction. Puis on additionne les deux entiers obtenus pour obtenir la somme en base 10. On convertit cette somme en chiffre romain (voir programme ex 2.3) et on affiche le résultat en chiffre romain.

Pour insérer du code Python dans ce fichier LaTeX, j'ai utilisée ce site pour comprendre comment faire : <https://borntocode.fr/latex-comment-inserer-et-customiser-du-code-source/>.

Discussion sur les chiffres romains

J'ai lu la page Wikipedia (le lien que vous nous avez fourni pour aller plus loin : https://fr.wikipedia.org/wiki/Discussion:Num%C3%A9ration_romaine).

On apprend que le système romain ne gère pas naturellement les fractions ou les nombres < 1 , et que tout ce qu'on fait en chiffres romains concerne des entiers ≥ 1 (cela n'impactera pas ce programme).

La numération romaine traditionnelle permet de représenter les nombres jusqu'à 3999. Par convention, le chiffre 4 s'écrit IV (5-1), 40 s'écrit XL et 400 s'écrit CD. Il n'existe pas de symbole officiel pour 5000, ce qui fixe une limite pratique pour cette numération. Cette limite est mentionnée par Georges Ifrah dans *Histoire universelle des chiffres* (tome 1, page 454).

Certaines discussions suggèrent que des variantes existaient : par exemple, des répétitions plus longues de symboles comme IIII pour 4 ou XXXX pour 40 ont pu être utilisées, mais il n'y a pas de preuve formelle que ce soit systématique.

Donc on peut aussi rajouter une condition pour que les III s'arrête à 3 ou à 4.

J'avoue qu'au départ j'avais un doute sur est-ce que je parlais sur une version avec ses règles ou non.

Je ne savais pas trop si Wikipedia était une source fiable sur ce point, d'autant plus que la page citée dans le cours est une page de *discussion* et non un article "officiel", les informations qui s'y trouvent ne sont donc pas nécessairement des affirmations vérifiées. Je me suis donc renseignée davantage sur le sujet.

Le premier site que j'ai consulté est <https://19eme.fr/chiffres-romains.html>. Sur ce site, on trouve un convertisseur de chiffres romains qui accepte uniquement les nombres de 1 à 3999, ce qui confirme déjà la limite classique (voir encadré rouge) :



FIGURE 1 – Convertisseur du site 19eme.fr, limité à 3999

Le site précise également les règles de la numération romaine, notamment la règle de répétition (voir encadré rouge) :

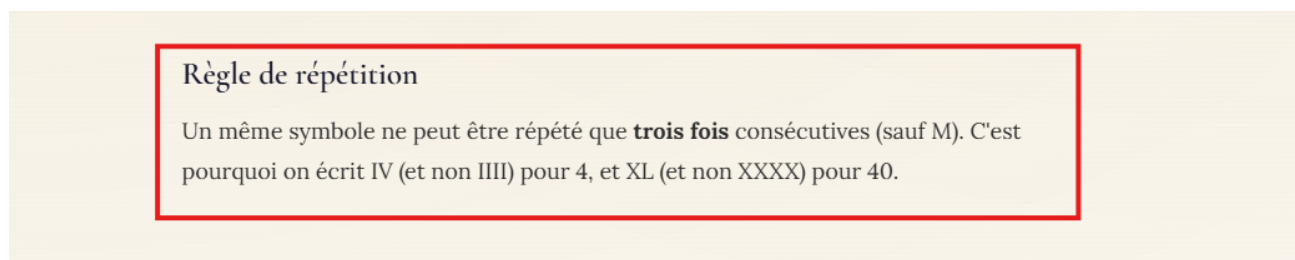


FIGURE 2 – Règle de répétition des symboles romains

Il est indiqué que "Un même symbole ne peut être répété que trois fois consécutives (sauf M). C'est pourquoi on écrit IV (et non IIII) pour 4, et XL (et non XXXX) pour 40." Cela va dans le sens de ce que disait la page Wikipedia citée dans le cours.

J'ai ensuite consulté le site <https://www.chiffre-en-romain.fr/faq>, qui répond directement à la question de la limite du système :

PEUT-ON CONVERTIR DE TRÈS GRANDS NOMBRES ? Y A-T-IL UNE LIMITE ?

Oui, mais avec une limite raisonnable : le système moderne s'arrête en général à 3 999. Au-delà, c'est techniquement possible mais pas vraiment standard, donc on préfère rester dans la zone lisible.

FIGURE 3 – FAQ du site chiffre-en-romain.fr sur la limite à 3999

Il y est indiqué que : *"le système moderne s'arrête en général à 3999. Au-delà, c'est techniquement possible mais pas vraiment standard, donc on préfère rester dans la zone lisible."*

Enfin, j'ai consulté <https://au-fil-des-pages.be/les-chiffres-romains-guide-complet-de-1-a-10000-en/>, qui précise que les nombres supérieurs à 3999 nécessitent des symboles étendus comme une barre horizontale au-dessus des lettres. Par exemple $\bar{M}$ pour un million. Ces symboles ne font pas partie du système romain "classique" et ne sont pas gérés par notre programme.

Ces trois sources convergent donc vers la même conclusion : dans le système romain classique, les nombres sont limités à 3999, un même symbole ne peut pas être répété plus de trois fois consécutivement pour I, X, C et M, et certains symboles comme V, L et D ne peuvent quant à eux jamais être répétés. On n'écrit jamais VV pour 10 par exemple, on écrit X. C'est pourquoi, malgré mon hésitation initiale, j'ai décidée de développer mon programme en intégrant ces trois vérifications, afin de respecter les conventions classiques de la numération romaine.

Je vais aussi utiliser `.upper()` qui est une méthode Python qui convertit automatiquement tous les caractères d'une chaîne en majuscules. Par exemple `"xiv".upper()` retourne `"XIV"` :

```
PS C:\Users\emma5\Downloads> python3
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> "xiv".upper()
'XIV'
```

FIGURE 4 – Exemple du `.upper()`

Cela permet de ne pas avoir à gérer les minuscules manuellement dans un dictionnaire. Donc plutôt que de dupliquer chaque entrée, on applique `.upper()` sur la saisie de l'utilisateur avant de la traiter et le dictionnaire n'aura besoin de contenir que les majuscules.

Explication du code

Voici ce que j'ai fait :

```
1 def romain_vers_entier(romain):
```

On définit une fonction qui prend en paramètre une chaîne de caractères représentant un nombre romain et retourne sa valeur entière en base 10.

```
1 romain = romain.upper()
```

On commence par convertir toute la saisie en majuscules grâce à `.upper()`. Cette méthode Python retourne une nouvelle chaîne avec tous les caractères en majuscules. Cela permet de ne plus avoir à gérer les minuscules manuellement dans le dictionnaire

```
1 valeurs = {  
2 'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000  
3 }
```

Grâce à `.upper()`, le dictionnaire ne contient que les majuscules, soit 7 entrées. C'est plus lisible et plus simple à maintenir.

```
1 repetables = {'I', 'X', 'C', 'M'}
```

On définit un ensemble contenant les symboles qui peuvent être répétés, mais au maximum 3 fois consécutivement. Les symboles V, L et D n'y figurent pas car ils ne peuvent jamais être répétés.

```
1 compteur = 1  
2 total = 0  
3 i = 0
```

On initialise trois variables : `compteur` suit le nombre de fois qu'un même symbole apparaît d'affilée, `total` accumule la valeur finale en base 10 et `i` est l'indice de parcours de la chaîne. On initialise `compteur` à 1 et non à 0 car le premier symbole compte déjà comme une occurrence.

```
1 while i < len(romain):  
2 lettre = romain[i]
```

On parcourt la chaîne caractère par caractère. On stocke le symbole courant dans `lettre`.

```
1 if lettre not in valeurs:  
2 print(f"Symbole romain invalide detecte : {lettre}")  
3 return None
```

On vérifie que le symbole saisi appartient bien au dictionnaire. Si ce n'est pas le cas, par exemple si l'utilisateur tape A ou 5, on affiche un message d'erreur et on retourne `None` pour interrompre la fonction.

```
1 if i > 0 and romain[i] == romain[i-1]:  
2 compteur += 1
```

Si le symbole courant est identique au précédent, on incrémente le compteur. On vérifie d'abord que `i > 0` pour ne pas accéder à `romain[-1]` lors du premier tour de boucle, ce qui donnerait un résultat incorrect.

```

1 if lettre in repetables and compteur > 3:
2     print(f"Erreur : le symbole {lettre} ne peut pas etre repete plus de 3 fois.")
3     return None
4 elif lettre not in repetables:
5     print(f"Erreur : le symbole {lettre} ne peut pas etre repete.")
6     return None

```

On applique les deux règles de répétition. Si le symbole fait partie des répétables (I, X, C, M) mais apparaît plus de 3 fois d'affilée, c'est une erreur. Si le symbole ne fait pas partie des répétables (V, L, D) et qu'il se répète, c'est également une erreur. Dans les deux cas on retourne `None`.

```

1 else:
2     compteur = 1

```

Si le symbole courant est différent du précédent, on remet le compteur à 1 pour repartir sur une nouvelle séquence.

```

1 if total > 3999:
2     print("Attention : la numeration romaine standard ne supporte pas les nombres > 3999")
3     return None

```

On vérifie à chaque tour que le total accumulé ne dépasse pas 3999.

```

1 if i + 1 < len(romain) and valeurs[lettre] < valeurs[romain[i + 1]]:
2     total += valeurs[romain[i + 1]] - valeurs[lettre]
3     i += 2
4 else:
5     total += valeurs[lettre]
6     i += 1

```

On vérifie d'abord que le symbole suivant existe pour éviter un `IndexError`, puis on détecte le cas soustractif. Si le symbole courant est plus petit que le suivant, on soustrait (ex : IV = 5 - 1 = 4) ; sinon on additionne. Dans les deux cas on avance dans la chaîne.

```

1 return total

```

Une fois tous les symboles parcourus et toutes les vérifications passées, on retourne la valeur entière obtenue.

```

1 romain1 = input("Entrez le premier nombre romain : ").upper()
2 romain2 = input("Entrez le second nombre romain : ").upper()

```

On applique `.upper()` directement sur la saisie de l'utilisateur, ce qui garantit que la chaîne est en majuscules avant même d'entrer dans la fonction.

```

1 entier1 = romain_vers_entier(romain1)
2 entier2 = romain_vers_entier(romain2)

```

On convertit les deux nombres romains en entiers via la fonction définie ci-dessus.

```
1 if entier1 is not None and entier2 is not None:
2     somme = entier1 + entier2
3     print(f"La somme en base 10 est : {somme}")
4     romain_resultat = entier_vers_romain(somme)
5     print(f"La somme de {romain1} et {romain2} en chiffres romains est : {romain_resultat}
6         ")
7 else:
8     print("Erreur lors de la conversion. Impossible de faire l'addition.")
```

On vérifie que les deux conversions ont réussi avant de calculer la somme. Si l'une des deux a retourné `None`, on affiche un message d'erreur. Sinon, on additionne les deux entiers, on affiche la somme en base 10, puis on appelle `entier_vers_romain()`, dont le fonctionnement est expliqué dans la section suivante (exercice 2.3).

Code final

Voici donc le code final :

```
1 """
2 /*****
3  * Nom      : Chap_2_ex_2.2_v2.py
4  * Role     : Convertir deux nombres romains en entiers, verifier leur
5  *           valide et afficher leur somme en base 10 et en chiffres romains
6  * Auteur   : Emma BARETS
7  * Version  : V2 du 14/12/2025
8  * Licence  : Realise dans le cadre du cours Architecture des ordinateurs
9  * Dependances : Aucun module externe requis
10 * Usage    : Entrer deux nombres romains et afficher leur somme en base 10
11 *           et en chiffres romains via la fonction de l'exercice 2.3
12 *****/
13 """
14 # Exercice 2.2 (Bonus)
15
16 def romain_vers_entier(romain):
17     """
18     Convertit un chiffre romain en entier.
19     Applique les regles classiques :
20     - Pas plus de 3 repetitions consecutives pour I, X, C, M
21     - V, L, D ne sont jamais repetes
22     - Limite de 3999 pour le resultat
23     """
24     # Conversion en majuscules
25     romain = romain.upper()
26
27     # Dictionnaire contenant la valeur de chaque symbole romain (majuscules
28     # uniquement grace a .upper())
29     valeurs = {
30         'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000
31     }
32
33     # Symboles autorises a etre repetes (maximum 3 fois consecutives)
34     # V, L, D sont absents car ils ne peuvent jamais etre repetes
35     repetables = {'I', 'X', 'C', 'M'}
```



```

36     compteur = 1 # Compteur de repetitions consecutives, initialise a 1 car le
        premier symbole compte deja
37     total = 0    # Accumulateur de la valeur finale en base 10
38     i = 0        # Indice de parcours de la chaine
39
40     # Parcours de chaque symbole du nombre romain
41     while i < len(romain):
42         lettre = romain[i] # Symbole courant
43
44         # Verification que le caractere est bien un symbole romain valide
45         if lettre not in valeurs:
46             print(f"Erreur : symbole invalide detecte : {lettre}")
47             return None
48
49         # Detection des repetitions consecutives
50         if i > 0 and romain[i] == romain[i-1]:
51             compteur += 1
52             # Symboles repetables : limite a 3 fois consecutives
53             if lettre in repetables and compteur > 3:
54                 print(f"Erreur : le symbole {lettre} ne peut pas etre repete plus de
                    3 fois consecutivement.")
55                 return None
56             # Symboles non repetables (V, L, D) : aucune repetition autorisee
57             elif lettre not in repetables:
58                 print(f"Erreur : le symbole {lettre} ne peut pas etre repete.")
59                 return None
60         else:
61             compteur = 1 # Reinitialisation du compteur si le symbole change
62
63         # Verification que le total ne depasse pas 3999
64         if total > 3999:
65             print("Erreur : la numeration romaine standard ne supporte pas les
                    nombres > 3999.")
66             return None
67
68         # Cas soustractif : le symbole courant est plus petit que le suivant
69         # On verifie d'abord que le symbole suivant existe pour eviter un IndexError
70         if i + 1 < len(romain) and valeurs[lettre] < valeurs[romain[i + 1]]:
71             total += valeurs[romain[i + 1]] - valeurs[lettre] # Ex : IV = 5 - 1 = 4
72             i += 2 # On saute les deux symboles utilises ensemble
73         # Cas additif : le symbole courant est superieur ou egal au suivant
74         else:
75             total += valeurs[lettre]
76             i += 1 # On passe au symbole suivant
77
78     return total # Valeur entiere en base 10
79
80 # On utilise la fonction de l'exercice 2.3
81
82 def entier_vers_romain(nombre):
83     """
84     Convertit un entier en chiffre romain.
85     """

```

```

86
87 # Verification que le nombre est strictement positif
88 if nombre <= 0:
89     print("La numeration romaine ne permet pas de représenter 0 ou les nombres
90         négatifs.")
91     return None
92
93 # Verification de la limite de la numeration romaine classique
94 # Par convention, les nombres supérieurs à 3999 ne sont pas représentés
95 if nombre > 3999:
96     print("La numeration romaine standard ne supporte pas les nombres > 3999")
97     return None
98
99 # Liste des symboles romains et de leur valeur associée
100 # On inclut les cas particuliers (CM, CD, XC, XL, IX, IV)
101 romains = [
102     ('M', 1000), ('CM', 900), ('D', 500), ('CD', 400),
103     ('C', 100), ('XC', 90), ('L', 50), ('XL', 40),
104     ('X', 10), ('IX', 9), ('V', 5), ('IV', 4), ('I', 1)
105 ]
106
107 # Chaîne vide qui contiendra progressivement le résultat final
108 resultat = ""
109
110 # Parcours de chaque symbole romain et de sa valeur associée
111 for symbole, valeur in romains:
112     # Tant que le nombre restant est supérieur ou égal
113     # à la valeur du symbole courant
114     while nombre >= valeur:
115
116         # Ajout du symbole romain au résultat
117         resultat += symbole
118
119         # Soustraction de la valeur utilisée
120         nombre -= valeur
121
122 # Retour du nombre converti en chiffres romains
123 return resultat
124
125
126 # Test du programme
127
128 # Demande à l'utilisateur de saisir un entier
129 nombre = int(input("Entrez un entier : "))
130
131 # Appel de la fonction de conversion
132 romain = entier_vers_romain(nombre)
133
134 # Verification que la conversion a fonctionné
135 if romain is not None:
136     # Affichage du résultat final
137     print(f"{nombre} en chiffres romains : {romain}")

```

```

138
139 else:
140     # Affichage d'un message d'erreur si la conversion est impossible
141     print("Conversion impossible.")
142
143 # Saisie et conversion en majuscules directement sur l'input
144 romain1 = input("Entrez le premier nombre romain : ").upper()
145 romain2 = input("Entrez le second nombre romain : ").upper()
146
147 # Conversion de chaque nombre romain en entier
148 entier1 = romain_vers_entier(romain1)
149 entier2 = romain_vers_entier(romain2)
150
151 # Verification que les deux conversions ont reussi avant de calculer
152 if entier1 is not None and entier2 is not None:
153     somme = entier1 + entier2
154     print(f"La somme en base 10 est : {somme}")
155     # Appel de la fonction de l'exercice 2.3 (voir section suivante)
156     romain_resultat = entier_vers_romain(somme)
157     print(f"La somme de {romain1} et {romain2} en chiffres romains est : {
        romain_resultat}")
158 else:
159     print("Erreur lors de la conversion des nombres romains. Impossible de faire l'
        addition.")

```

Tests

On passe maintenant aux tests.

Je teste avec deux nombres simples V et X avec la commande `python3 Chap_2_ex_2.2.py` :

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparati
on_envoi_chap_2_a_5\Annexe> python3 Chap_2_ex_2.2.py
Entrez le premier nombre romain : V
Entrez le second nombre romain : X
La somme en base 10 est : 15
La somme de V et X en chiffres romains est : XV
```

FIGURE 5 – Test du programme avec V et X

J'obtiens bien 15 et la somme de V et X en chiffres romains est bien XV.

Ensuite, je teste avec iv et i, pour vérifier si le cas négatif et les minuscules fonctionnent :

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparati
on_envoi_chap_2_a_5\Annexe> python3 Chap_2_ex_2.2.py
Entrez le premier nombre romain : iv
Entrez le second nombre romain : i
La somme en base 10 est : 5
La somme de IV et I en chiffres romains est : V
```

FIGURE 6 – Test du programme avec iv et i

J'ai bien 5 et la somme de IV et I est V.

Dernier test, je teste avec deux grands nombres (qui ne dépassent pas 3999) :

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparati
on_envoi_chap_2_a_5\Annexe> python3 Chap_2_ex_2.2.py
Entrez le premier nombre romain : MCMXCIX
Entrez le second nombre romain : CD
La somme en base 10 est : 2399
La somme de MCMXCIX et CD en chiffres romains est : MMCCCXCIX
```

FIGURE 7 – Test du programme avec deux grands nombres

Cette fois-ci, pour être sûr, j'ai utilisée le site que vous nous avez conseillé dans "Pour aller plus loin" (WolframAlpha) : <https://www.wolframalpha.com/>

Dans la barre de recherche du site, on peut taper $\text{MCMXCIX} + \text{CD}$:

FROM THE MAKERS OF WOLFRAM LANGUAGE AND MATHEMATICA

WolframAlpha

NATURAL LANGUAGE

MATH INPUT

EXTENDED KEYBOARD

EXAMPLES

UPLOAD

RANDOM

Assuming "CD" is a roman numeral | Use as a unit instead

Input interpretation:

MCMXCIX (Roman numerals) + CD (Roman numerals)

Result:

MMCCCXCIX

Decimal form:

2399

Other antique number systems:

Mayan

Babylonian

Greek

,βτϞθ

FIGURE 8 – Test avec $\text{MCMXCIX} + \text{CD}$ sur le site

On peut voir, dans la section "Result" (Résultat), que $\text{MCMXCIX} + \text{CD} = \text{MMCCCXCIX}$. J'ai entourée en bleu, sur la capture d'écran, la zone correspondante. On voit aussi, dans la section "Decimal form" (Forme décimale), que $\text{MCMXCIX} + \text{CD} = 2399$. J'ai entourée en rouge, sur la capture d'écran, la zone correspondante. Cela confirme que notre programme marche.

Maintenant, on va vérifier si notre gestion d'erreur et nos règles.

On commence par tester si on rentre IV et DD :

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparati
on_envoie_chap_2_a_5\Annexe> python3 Chap_2_ex_2.2.py
Entrez le premier nombre romain : IV
Entrez le second nombre romain : DD
Erreur : le symbole D ne peut pas etre repete.
Erreur lors de la conversion des nombres romains. Impossible de faire l'addition.
```

FIGURE 9 – Test répétitions 1

On a bien les messages suivant qui s'affiche : "Erreur : le symbole D ne peut pas etre repete." et "Erreur lors de la conversion des nombres romains. Impossible de faire l'addition."

On essaye avec IIII et I :

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparati
on_envoie_chap_2_a_5\Annexe> python3 Chap_2_ex_2.2.py
Entrez le premier nombre romain : IIII
Entrez le second nombre romain : I
Erreur : le symbole I ne peut pas etre repete plus de 3 fois consecutivement.
Erreur lors de la conversion des nombres romains. Impossible de faire l'addition.
```

FIGURE 10 – Test répétitions 2

On a bien : "Erreur : le symbole I ne peut pas etre repete plus de 3 fois consecutivement." et "Erreur lors de la conversion des nombres romains. Impossible de faire l'addition."

Un dernier test avec A et 2 :

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparati
on_envoie_chap_2_a_5\Annexe> python3 Chap_2_ex_2.2.py
Entrez le premier nombre romain : A
Entrez le second nombre romain : 2
Erreur : symbole invalide detecte : A
Erreur : symbole invalide detecte : 2
```

FIGURE 11 – Test avec des caractères incorrectes

On a bien : "Erreur : symbole invalide detecte : A", "Erreur : symbole invalide detecte : 2" et "Erreur lors de la conversion des nombres romains. Impossible de faire l'addition."

Les codes seront disponibles en annexe.

Conclusion

Au départ, j'avais développé une première version du programme qui fonctionnait pour les cas classiques donc elle convertissait correctement les chiffres romains en entiers et calculait leur somme. Cependant, en me renseignant davantage sur les règles de la numération romaine, je me suis rendu compte qu'elle avait des lacunes : elle acceptait des entrées invalides comme IIII ou VV sans signaler d'erreur, elle ne bloquait pas les nombres supérieurs à 3999 et on pouvait rentrer des caractères invalide comme A. En d'autres termes, elle produisait un résultat sans planter mais ce résultat pouvait être incorrect sans que l'utilisateur s'en rende compte. La version présentée ici corrige ces problèmes en ajoutant des vérifications sur la validité de l'entrée.

Cet exercice était relativement accessible, le code en lui-même ne m'a pas posé de difficultés majeures.

Enfin, cet exercice m'a aussi confronté à une question que je me pose encore. En cours au lycée, on n'avait pas le droit d'utiliser des fonctions déjà définies comme `sum()` sans les avoir recodées soi-même au préalable. À la fac, je ne sais pas encore exactement où se situe la limite. Dans le sens où est-ce qu'on peut utiliser librement ce que Python propose tant qu'on l'explique et qu'on cite ses sources ou est-ce qu'il faut tout reconstruire ? C'est quelque chose auquel je dois me détacher et être un peu plus indépendante de cela.

1.2 Exercice 2.3 (A Rendre)

Réaliser un programme, dans le langage de votre choix (cf Petit Guide), qui imprime un nombre saisi par l'utilisateur en chiffres romains.

Pour vous aider : Voir les liens de la section "Pour aller (Encore) plus loin".

Explications de mon code

Voici ce que j'ai fait :

```
1 def entier_vers_roman(nombre):
```

Je définis une fonction qui prend en entrée un entier en base 10 et qui le convertit en chiffres romains.

```
1 if nombre <= 0:
```

Je vérifie que le nombre est strictement positif.

```
1 print("La numération romaine ne permet pas de représenter 0 ou les nombres négatifs.")
   )
2 return None
```

Si le nombre vaut 0 ou est négatif, j'affiche un message d'erreur puis j'arrête la fonction. En effet, la numération romaine classique ne possède pas de symbole pour représenter le zéro et ne permet pas non plus de représenter les nombres négatifs.

```
1 if nombre > 3999:
```

Je vérifie ensuite que le nombre ne dépasse pas 3999.

```
1 print("La numération romaine standard ne supporte pas les nombres > 3999")
2 return None
```

Si le nombre est supérieur à 3999, j'affiche un message d'erreur puis j'interromps la conversion. En effet, la numération romaine standard est généralement limitée à 3999.

```
1 romains = [
2     ('M', 1000), ('CM', 900), ('D', 500), ('CD', 400),
3     ('C', 100), ('XC', 90), ('L', 50), ('XL', 40),
4     ('X', 10), ('IX', 9), ('V', 5), ('IV', 4), ('I', 1)
5 ]
```

Je crée une liste contenant chaque symbole romain ainsi que sa valeur en base 10. Les symboles sont classés du plus grand au plus petit afin de faciliter la conversion.

```
1 resultat = ""
```

Je crée une chaîne de caractères vide qui contiendra progressivement le résultat final en chiffres romains.

```
1 for symbole, valeur in romains:
```

Je parcours chaque couple composé d'un symbole romain et de sa valeur associée.

```
1 while nombre >= valeur:
```

Tant que le nombre restant est supérieur ou égal à la valeur du symbole courant, je peux utiliser ce symbole dans la conversion.

```
1 resultat += symbole
```

J'ajoute le symbole romain au resultat.

```
1 nombre -= valeur
```

Je soustrais la valeur correspondante du nombre restant a convertir.

```
1 return resultat
```

Une fois toutes les conversions effectuées, la fonction retourne le resultat final en chiffres romains.

Ensuite, je réalise quelques tests afin de verifier le bon fonctionnement du programme.

```
1 nombre = int(input("Entrez un entier : "))
```

Je demande a l'utilisateur de saisir un nombre entier.

```
1 romain = entier_vers_romain(nombre)
```

J'appelle la fonction de conversion afin d'obtenir l'écriture du nombre en chiffres romains.

```
1 if romain is not None:
```

Je vérifie que la conversion s'est bien déroulée.

```
1 print(f"{nombre} en chiffres romains : {romain}")
```

Si la conversion a réussi, j'affiche le résultat obtenu.

```
1 else:  
2     print("Conversion impossible.")
```

Si la conversion n'a pas pu etre effectuée, j'affiche un message d'erreur.

Code finale

```
1 """
2 /*****
3 * Nom      : Chap_2_ex_2.3.py
4 * Role     : Convertir un entier en chiffres romains (jusqu'a 3999)
5 * Auteur   : Emma BARETS
6 * Version  : Version finale du 14/12/2025
7 * Licence  : Realise dans le cadre du cours Architecture des ordinateurs
8 * Dependances : Aucun module externe requis
9 * Usage    : Entrer un entier et obtenir son ecriture en chiffres romains
10 *****/
11 """
12
13 def entier_vers_roman(nombre):
14     """
15     Convertit un entier en chiffre romain.
16     """
17
18     # Verification que le nombre est strictement positif
19     if nombre <= 0:
20         print("La numeration romaine ne permet pas de représenter 0 ou les nombres
21             negatifs.")
22         return None
23
24     # Verification de la limite de la numeration romaine classique
25     # Par convention, les nombres superieurs a 3999 ne sont pas representes
26     if nombre > 3999:
27         print("La numeration romaine standard ne supporte pas les nombres > 3999")
28         return None
29
30     # Liste des symboles romains et de leur valeur associee
31     # On inclut les cas particuliers (CM, CD, XC, XL, IX, IV)
32     romains = [
33         ('M', 1000), ('CM', 900), ('D', 500), ('CD', 400),
34         ('C', 100), ('XC', 90), ('L', 50), ('XL', 40),
35         ('X', 10), ('IX', 9), ('V', 5), ('IV', 4), ('I', 1)
36     ]
37
38     # Chaine vide qui contiendra progressivement le resultat final
39     resultat = ""
40
41     # Parcours de chaque symbole romain et de sa valeur associee
42     for symbole, valeur in romains:
43         # Tant que le nombre restant est superieur ou egal
44         # a la valeur du symbole courant
45         while nombre >= valeur:
46
47             # Ajout du symbole romain au resultat
48             resultat += symbole
49
50             # Soustraction de la valeur utilisee
```

```

51         nombre -= valeur
52
53     # Retour du nombre converti en chiffres romains
54     return resultat
55
56
57 # Test du programme
58
59 # Demande a l'utilisateur de saisir un entier
60 nombre = int(input("Entrez un entier : "))
61
62 # Appel de la fonction de conversion
63 romain = entier_vers_romain(nombre)
64
65 # Verification que la conversion a fonctionne
66 if romain is not None:
67     # Affichage du resultat final
68     print(f"{nombre} en chiffres romains : {romain}")
69
70 else:
71     # Affichage d'un message d'erreur si la conversion est impossible
72     print("Conversion impossible.")

```

Tests

Je test avec 32, pour ce faire, je retourne sur le site Wolframalpha :

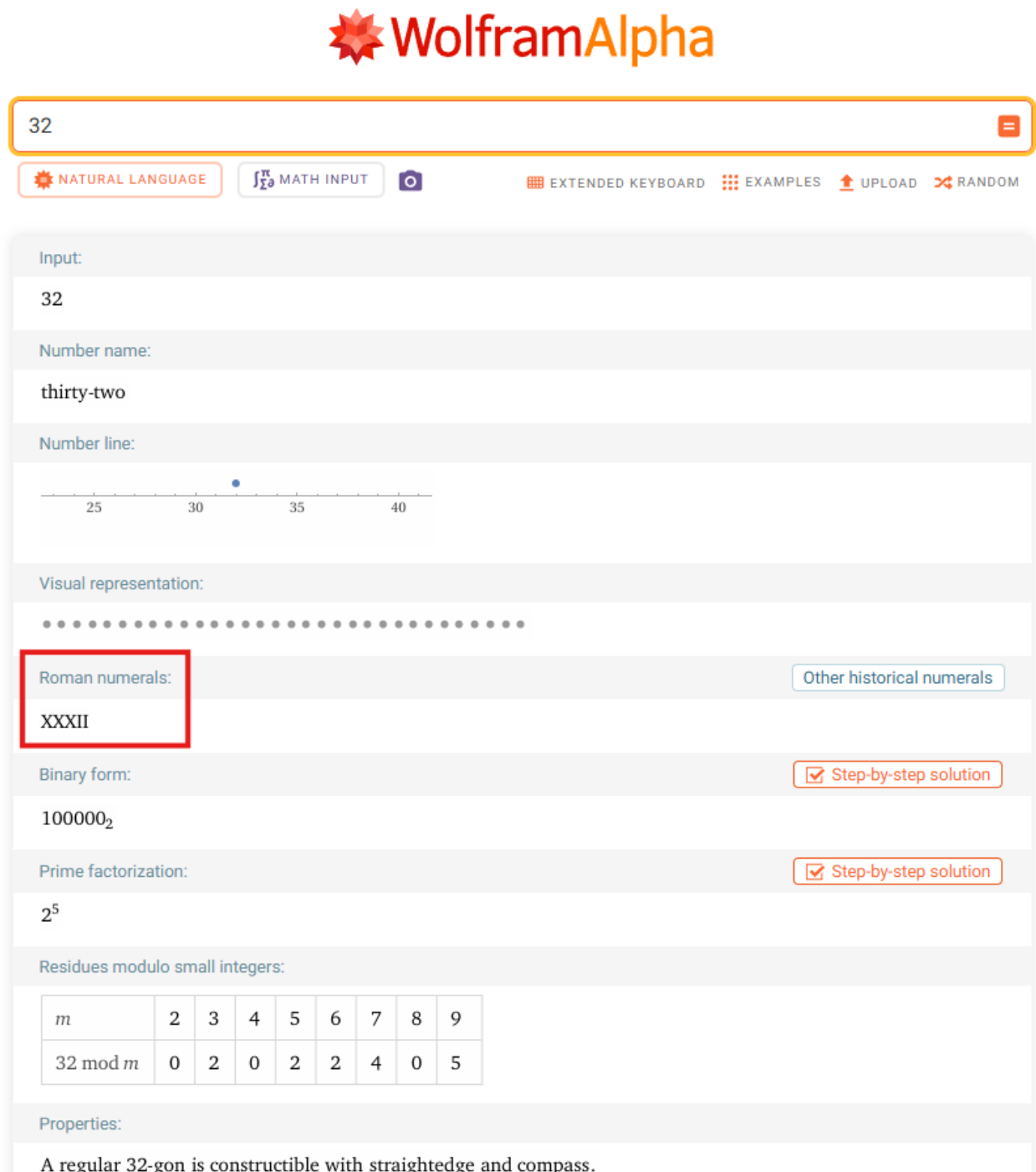


FIGURE 12 – Test avec 32

On peut voir, dans la section "Roman numerals" (chiffres romains), que 32 s'écrit XXXII en chiffres romains. J'ai entourée en rouge, sur la capture d'écran, la zone correspondante.

Je teste maintenant mon programme avec mon code.
 Dans le terminal, j'exécute la commande `python3 Chap_2_ex_2.3`.
 Je saisis la valeur 32 :

```
PS C:\Users\emma5\Downloads> python3 Chap_2_ex_2.3.py
Entrez un entier : 32
32 en chiffres romains : XXXII
```

FIGURE 13 – Test avec 32

Ici, j'obtiens bien le résultat XXXII.
 Je test avec 230, pour ce faire, je retourne sur le site WolframAlpha :

FROM THE MAKERS OF WOLFRAM LANGUAGE AND MATHEMATICA

WolframAlpha

230

NATURAL LANGUAGE
MATH INPUT
EXTENDED KEYBOARD
EXAMPLES
UPLOAD
RANDOM

Input:

230

Number name:

two hundred thirty

Number line:

Roman numerals:

CCXXX

Other historical numerals

Binary form:

11100110₂

Prime factorization:

2 × 5 × 23

Residues modulo small integers:

m	2	3	4	5	6	7	8	9
230 mod m	0	2	2	0	2	6	6	5

Properties:

FIGURE 14 – Test avec 230

On peut voir, dans la section "Roman numerals" (chiffres romains), que 230 s'écrit CCXXX en chiffres romains. J'ai entourée en rouge, sur la capture d'écran, la zone correspondante.

Je teste maintenant mon programme avec mon code.
 Dans le terminal, j'exécute la commande `python3 Chap_2_ex_2.3`.
 Je saisis la valeur 230 :

```
PS C:\Users\emma5\Downloads> python3 Chap_2_ex_2.3.py
Entrez un entier : 230
230 en chiffres romains : CCXXX
```

FIGURE 15 – Test avec 230

J'ai bien CCXXX.
 Dernier test pour être sur, je test avec 3283, pour ce faire, je retourne sur le site WolframAlpha :

WolframAlpha

3283

NATURAL LANGUAGE MATH INPUT

EXTENDED KEYBOARD EXAMPLES UPLOAD RANDOM

Assuming "3283" is a decimal number | Use as a musical scale instead

Input:

3283

Number name: Words and numerals

three thousand, two hundred eighty-three

Number line:

2500 3000 3500 4000

Roman numerals: Other historical numerals

MMMCLXXXIII

Binary form: Step-by-step solution

110011010011₂

Prime factorization: Step-by-step solution

7² × 67

Residues modulo small integers:

m	2	3	4	5	6	7	8	9
3283 mod m	1	1	3	3	1	0	3	7

Properties:

FIGURE 16 – Test avec 3283

On peut voir, dans la section "Roman numerals" (chiffres romains), que 3283 s'écrit MMMCLXXXIII en

chiffres romains. J'ai entourée en rouge, sur la capture d'écran, la zone correspondante.

Je teste maintenant mon programme avec mon code.

Dans le terminal, j'exécute la commande `python3 Chap_2_ex_2.3`.

Je saisis la valeur 3283 :

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparation_envoie_chap_2_a_5\Annexe> python3 Chap_2_ex_2.3.py
Entrez un entier : 3283
3283 en chiffres romains : MMMCCLXXXIII
```

FIGURE 17 – Test avec 3283

J'ai bien MMMCCLXXXIII.

On va tester une erreur qui doit être gérée par le programme, par exemple avec un nombre supérieur à 3999 :

```
PS C:\Users\emma5\Downloads> python3 Chap_2_ex_2.3.py
Entrez un entier : 5000
La numeration romaine standard ne supporte pas les nombres > 3999
Conversion impossible.
```

FIGURE 18 – Test avec un nombre > à 3999

J'ai bien : "La numeration romaine standard ne supporte pas les nombres > 3999. Conversion impossible."

Conclusion

Cet exercice demandait de faire l'inverse de l'exercice précédent. Autrement dit, il faut prendre un entier et afficher sa représentation en chiffres romains. Pour ça, j'ai écrit une fonction qui parcourt une liste de symboles du plus grand au plus petit et qui soustrait chaque valeur au nombre restant jusqu'à arriver à zéro. En prenant en compte les règles que l'on a vu dans l'exercice précédent.

Comme pour l'exercice précédent, j'ai découvert que la numération romaine : ne gère pas les fractions ni les nombres inférieurs à 1, elle est limitée par convention à 3999 car il n'existe pas de symbole officiel pour 5000 par exemple et certaines lettres comme V, L et D ne peuvent jamais être répétées, tandis que I, X, C et M sont limitées à trois répétitions consécutives. Ces règles m'ont semblé suffisamment importantes pour les intégrer dans le code.

1.3 Exercice 2.12 (A Rendre)

Convertir le nombre $(3391)_{10}$ en base -5. (Facile si vous avez déjà répondu au 2.11)

Rappel : les restes des divisions par -5 doivent être positifs. Vous ne devez pas utiliser la forme (par exemple) $18 = 3 \ 5 \ 3$ mais plutôt $18 = 4 \cdot 5 + 2$.

Réponse :

Pour convertir le nombre 3391 en base -5, on fait :

1) 3391 divisé par -5 = -678 avec un reste de 1.

On écrit donc : $3391 = (-678) \cdot (-5) + 1$

2) -678 divisé par -5 = 136 avec un reste de 2.

On écrit : $-678 = 136 \cdot (-5) + 2$

3) 136 divisé par -5 = -27 avec un reste de 1.

On écrit : $136 = (-27) \cdot (-5) + 1$

4) -27 divisé par -5 = 6 avec un reste de 3.

On écrit : $-27 = 6 \cdot (-5) + 3$

5) 6 divisé par -5 = -1 avec un reste de 1.

On écrit : $6 = (-1) \cdot (-5) + 1$

6) -1 divisé par -5 = 1 avec un reste de 4.

On écrit : $-1 = 1 \cdot (-5) + 4$

7) 1 divisé par -5 = 0 avec un reste de 1.

On écrit : $1 = 0 \cdot (-5) + 1$

Ensuite, on lit tous les restes de bas en haut pour obtenir le nombre en base -5.

Donc, $(3391)_{10} = (1413121)_{-5}$.

Conclusion

Cet exercice demandait de convertir le nombre 3391 en base -5. Le principe est le même que pour une conversion en base positive c'est-à-dire qu'on divise successivement par la base et on relève les restes à chaque étape jusqu'à obtenir un quotient nul. La seule différence ici est que la base est négative, ce qui demande quelques ajustements. Je me suis entraîné grâce aux exercices "Au Choix".

J'ai compris que :

- les restes des divisions par -5 doivent toujours être positifs, c'est-à-dire appartenir à $\{0, 1, 2, 3, 4\}$;
- quand le reste obtenu est négatif, il faut ajuster : par exemple, si on obtient $-1 = 3 \times (-5) - 3$, ce n'est pas valide. On réécrit plutôt $-1 = 1 \times (-5) + 4$, en ajustant le quotient en conséquence ;
- on arrête les divisions uniquement quand le quotient est égal à zéro et pas avant (j'avais justement oublié une étape dans mon premier calcul à cause de ça) ;
- une fois toutes les étapes effectuées, on lit les restes de bas en haut pour obtenir la représentation finale en base -5.

1.4 Conclusion du chapitre 2

Ce chapitre portait sur la représentation des nombres entiers positifs. C'est un chapitre qui m'a semblée globalement accessible avec des notions qui se comprennent bien une fois qu'on a assimilé la logique de base. Les exercices sur les chiffres romains m'ont aussi permis d'apprendre des choses que je ne savais pas comme les règles de répétition des symboles ou la limite à 3999. Dans l'ensemble, c'est un chapitre assez intéressant.

2 Sources du chapitre 2

- Kislin-Duval, P. (2022). *Architecture des Machines et des Ordinateurs : Support de Cours* (Version 1.2). Institut d'Enseignement à Distance, Licence Informatique.
- Python-3.com. (s. d.). *La méthode .upper()* <https://fr.python-3.com/?p=2788>
- BornToCode.fr. (2026). *LaTeX : comment insérer et customiser du code source* <https://borntocode.fr/latex-comment-inserer-et-customiser-du-code-source/>
- WolframAlpha. (s. d.). *WolframAlpha : Computational intelligence* <https://www.wolframalpha.com/>
- Wikipédia. (2023). *Discussion : Numération romaine* https://fr.wikipedia.org/wiki/Discussion:Num%C3%A9ration_romaine
- Ifrah, G. (1998). *Histoire universelle des chiffres* (tome 1, p. 454). Robert Laffont.
- 19eme.fr. (s. d.). *Les chiffres romains*. <https://19eme.fr/chiffres-romains.html>
- Au fil des pages. (2025). *Les chiffres romains : guide complet de 1 à 10000*. <https://au-fil-des-pages.be/les-chiffres-romains-guide-complet-de-1-a-10000-en/>
- Chiffre-en-romain.fr. (s. d.). *FAQ sur les chiffres romains*. <https://www.chiffre-en-romain.fr/faq>

Chapitre 3 — Opérations en Binaire & Représentation des Nombres Signés

Une première Série d'Exercices

Dans cette première série d'exercice, nous vous invitons à les réaliser sur au brouillon. Ce sont des exercices d'entraînement. Attention, l'exercice 3.6 sera à rendre (voir liste plus loin).

2.1 Exercice 3.6 (A Rendre)

La base -2 n'utilise que les chiffres 0 et 1. (Facile) Traduire en décimal les seize valeurs entre $(0000)_{-2}$ et $(1111)_{-2}$. (Plus difficile) Quelle est la table d'addition de la base -2 ?

Pour vous aider : voir l'article "Système négabinaire" sur Wikipédia...

Réponse :

Partie 1 — Conversion des seize valeurs en décimal

Pour représenter un nombre en base -2 , on multiplie chaque bit par la puissance correspondante de la base, et on fait la somme. Pour un nombre à n bits $b_{n-1} \cdots b_1 b_0$, la formule est donc :

$$N = \sum_{i=0}^{n-1} b_i \cdot (-2)^i$$

En base +2 classique, toutes les puissances sont positives, donc on ne peut représenter que des entiers positifs. Ici, les puissances alternent : $(-2)^0 = 1$, $(-2)^1 = -2$, $(-2)^2 = 4$, $(-2)^3 = -8$. Le fait que $(-2)^3$ soit négatif permet d'atteindre des valeurs négatives sans avoir besoin d'un bit de signe séparé : il suffit que le bit de gauche soit à 1.

Pour 1001_{-2} par exemple, on calcule :

$$1 \cdot (-2)^3 + 0 \cdot (-2)^2 + 0 \cdot (-2)^1 + 1 \cdot (-2)^0 = -8 + 0 + 0 + 1 = -7$$

On applique cette formule aux 16 combinaisons possibles de 4 bits, de 0000_{-2} à 1111_{-2} , ce qui donne le tableau suivant :

Base -2	Décimal	Calcul
0000	0	0
0001	1	1
0010	-2	-2
0011	-1	$-2 + 1$
0100	4	4
0101	5	$4 + 1$
0110	2	$4 + (-2)$
0111	3	$4 + (-2) + 1$
1000	-8	-8
1001	-7	$-8 + 1$
1010	-10	$-8 + (-2)$
1011	-9	$-8 + (-2) + 1$
1100	-4	$-8 + 4$
1101	-3	$-8 + 4 + 1$
1110	-6	$-8 + 4 + (-2)$
1111	-5	$-8 + 4 + (-2) + 1$

En binaire classique sur 4 bits, on ne peut représenter que les entiers de 0 à 15 — tous positifs. Pour représenter des négatifs, il faudrait un système supplémentaire comme le complément à 2. Ici, la base -2 donne "naturellement" les 16 valeurs $\{-10, -9, -8, -7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5\}$. On remarque aussi que les valeurs ne sont pas dans l'ordre croissant : par exemple $0001_{-2} = 1$ mais $0010_{-2} = -2$, donc passer d'une représentation à la suivante ne fait pas forcément augmenter la valeur décimale.

Partie 2 — Table d'addition en base -2

On commence par les cas simples où aucune retenue n'arrive dans la colonne.

a_i	+	b_i	$= c_i$
0	+	0	$= 0$
0	+	1	$= 1$
1	+	0	$= 1$
1	+	1	$= 110_{-2}$

En pratique, chaque colonne peut aussi recevoir une retenue. Il faut donc traiter tous les cas de $r_i + a_i + b_i$:

r_i	+	a_i	+	b_i	$= c_i$
-1	+	0	+	0	$= 111_{-2}$
0	+	0	+	0	$= 0$
+1	+	0	+	0	$= 1$
-1	+	0	+	1	$= 0$
0	+	0	+	1	$= 1$
+1	+	0	+	1	$= 110_{-2}$
-1	+	1	+	0	$= 0$
0	+	1	+	0	$= 1$
+1	+	1	+	0	$= 110_{-2}$
-1	+	1	+	1	$= 1$
0	+	1	+	1	$= 110_{-2}$
+1	+	1	+	1	$= 111_{-2}$

Conclusion

Cet exercice demandait de traduire en décimal les seize valeurs entre $(0000)_{-2}$ et $(1111)_{-2}$, puis de construire la table d'addition en base -2 .

Ce qui était nouveau pour moi dans cet exercice c'est de travailler avec une base -2 , où les puissances alternent entre valeurs positives et négatives.

J'ai compris que :

- la méthode de conversion reste la même que pour n'importe quelle base, même quand la base est négative : on multiplie chaque chiffre par la puissance correspondante et on somme ;
- les puissances de -2 alternent entre valeurs positives et négatives $(1, -2, 4, -8, 16, -32 \dots)$, ce qui fait que les valeurs décimales obtenues ne sont pas rangées dans le même ordre qu'en binaire "classique" ;
- pour la table d'addition, le seul cas particulier est $1 + 1$ qui donne 0 avec une retenue de -1 , car $2 = 0 + (-2) \times (-1)$ et les autres cas restent identiques à une addition binaire classique.

Une Seconde Série d'Exercices...

Pour ce chapitre trois, vous devez rendre les exercices **3.6** et **3.14**. L'exercice **3.15** est pour un Bonus.

2.2 Exercice 3.14 (A Rendre)

Étant donné deux nombres représentés en complément à 2, comment peut-on savoir lequel est le plus grand en n'utilisant que des comparaisons de bits entre eux ?

Écrire un programme dans le langage de votre choix pour effectuer cette comparaison. (Avec ou sans saisie de ces nombres par l'utilisateur selon vos connaissances dans le langage choisi)

Pour vous aider : Si les bits de signe sont différents, c'est le nombre où il vaut 0 qui est le plus grand...

On cherche à déterminer lequel de deux nombres représentés en complément à 2 est le plus grand, uniquement en comparant leurs bits. En complément à 2, le bit le plus à gauche, appelé bit de signe, indique si le nombre est positif (0) ou négatif (1).

Le programme commence par convertir les deux nombres en binaire sur un nombre suffisant de bits afin d'obtenir leur représentation complète. Cela permet de gérer correctement à la fois les nombres positifs et négatifs. Ensuite, il récupère le bit de signe de chaque nombre.

Si les bits de signe sont différents, le nombre dont le bit de signe vaut 0 est forcément le plus grand, car il est positif. Si les deux nombres ont le même signe, le programme compare les bits restants un par un, de gauche à droite. Pour les nombres positifs, un bit 1 est plus grand qu'un bit 0. Pour les nombres négatifs, c'est l'inverse : plus la valeur binaire est grande, plus le nombre réel est petit.

Enfin, si tous les bits sont identiques, les deux nombres sont égaux.

Explication du code

```
1 a = int(input("Entrez le premier nombre : "))
2 b = int(input("Entrez le deuxieme nombre : "))
```

Je demande à l'utilisateur de saisir deux nombres en base décimale, puis je les convertis en entiers.

```
1 max_val = max(abs(a), abs(b))
2 bits = 1
3 while (1 << (bits - 1)) <= max_val:
4     bits += 1
```

Je calcule la valeur absolue maximale entre les deux nombres pour déterminer le nombre de bits nécessaire.

Je commence à 1 bit, puis j'augmente jusqu'à ce que ce soit suffisant pour représenter les deux nombres en complément à 2.

```
1 def to_complement2(n, bits):
2     if n < 0:
3         n = (1 << bits) + n
4     return n
```

Je convertis un entier en complément à 2.

Si le nombre est négatif, j'ajoute 2^{bits} pour obtenir sa représentation binaire.

```
1 a_bin = to_complement2(a, bits)
2 b_bin = to_complement2(b, bits)
```

Je convertis les deux nombres en complément à 2.

```
1 def bit_signe(n, bits):
2     if n >= (1 << (bits - 1)):
3         return 1
4     else:
5         return 0
```

Je récupère le bit de signe : 0 = positif, 1 = négatif.

```
1 signe_a = bit_signe(a_bin, bits)
2 signe_b = bit_signe(b_bin, bits)
```

Je récupère les bits de signe des deux nombres.

```
1 if signe_a != signe_b:
```

Si les signes sont différents, le nombre positif est forcément le plus grand.

```
1     if signe_a == 0:
2         print(f"{a} est plus grand que {b}")
3     else:
4         print(f"{b} est plus grand que {a}")
```

J'affiche directement le résultat selon le signe.

```
1 else:
```

Sinon les deux nombres ont le même signe.

```
1     if a_bin > b_bin:
2         print(f"{a} est plus grand que {b}")
3     elif a_bin < b_bin:
4         print(f"{b} est plus grand que {a}")
5     else:
6         print(f"{a} et {b} sont égaux")
```

Une fois que les deux nombres ont été convertis en complément à 2 sur le même nombre de bits, il suffit de comparer directement leurs valeurs binaires.

Si la valeur binaire de a est supérieure à celle de b, alors a est plus grand.

Si elle est inférieure, alors b est plus grand.

Sinon, les deux nombres sont égaux.

Code final

Voici le programme (également disponible dans l'annexe voir fichier README) :

```
1 """
2 /*****
3  * Nom      : Chap_3_ex_3.14.py
4  * Role     : Comparer deux nombres decimaux en utilisant la representation
5  *           en complement a 2 et determiner le nombre de bits necessaires
6  * Auteur   : Emma BARETS
7  * Version  : Version finale du 12/01/2026
8  * Licence  : Realise dans le cadre du cours Architecture des ordinateurs
9  * Dependances : Aucun module externe requis
10 * Usage     : Executer le script et saisir deux nombres pour obtenir le plus grand
11 *****/
12 """
13
14 # Saisie des nombres en decimal
15 a = int(input("Entrez le premier nombre : "))
16 b = int(input("Entrez le deuxieme nombre : "))
17
18 # Calcul du nombre minimum de bits pour représenter les deux nombres
19 max_val = max(abs(a), abs(b))
20 bits = 1
21 while (1 << (bits - 1)) <= max_val:
22     bits += 1
23
24 print(f"Nombre de bits utilise : {bits}")
25
26 # Fonction pour convertir un nombre en complement a 2 sur 'bits' bits
27 def to_complement2(n, bits):
28     if n < 0:
29         n = (1 << bits) + n # ajouter 2^bits si le nombre est negatif
30     return n
31
32 # Conversion des nombres en complement a 2
33 a_bin = to_complement2(a, bits)
34 b_bin = to_complement2(b, bits)
35
36 # Determiner le bit de signe (0 = positif, 1 = negatif)
37 def bit_signe(n, bits):
38     if n >= (1 << (bits - 1)):
39         return 1
40     else:
41         return 0
42
43 signe_a = bit_signe(a_bin, bits)
44 signe_b = bit_signe(b_bin, bits)
45
46 # Comparaison selon le signe
47 if signe_a != signe_b:
48     if signe_a == 0:
49         print(f"{a} est plus grand que {b}")
```

```

50     else:
51         print(f"{b} est plus grand que {a}")
52 else:
53     # Meme signe : comparer les valeurs en complement a 2
54     if a_bin > b_bin:
55         print(f"{a} est plus grand que {b}")
56     elif a_bin < b_bin:
57         print(f"{b} est plus grand que {a}")
58     else:
59         print(f"{a} et {b} sont egaux")

```

Tests

Voici quelques tests pour verifier que le programme fonctionne correctement :

Test avec a et b positifs, et $b > a$:

```

>>> %Run 'chap 3.py'

Entrez le premier nombre : 3
Entrez le deuxieme nombre : 6
Nombre de bits utilise : 4
6 est plus grand que 3

```

FIGURE 19 – Test a et b positifs et $b > a$

Test avec a et b positifs, et $a > b$:

```

>>> %Run 'chap 3.py'

Entrez le premier nombre : 6
Entrez le deuxieme nombre : 3
Nombre de bits utilise : 4
6 est plus grand que 3

```

FIGURE 20 – Test avec a et b positifs et $a > b$

Test avec a negatif et b positif :

```

>>> %Run 'chap 3.py'

Entrez le premier nombre : -2
Entrez le deuxieme nombre : 9
Nombre de bits utilise : 5
9 est plus grand que -2

```

FIGURE 21 – Test avec a negatif et b positif

Test avec a positif et b negatif :

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : 100
Entrez le deuxieme nombre : -4
Nombre de bits utilise : 8
100 est plus grand que -4
```

FIGURE 22 – Test avec a positif et b negatif

Test avec a et b negatifs :

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : -4
Entrez le deuxieme nombre : -93
Nombre de bits utilise : 8
-4 est plus grand que -93
```

FIGURE 23 – Test avec a et b negatifs

Maintenant, on peut tester un cas limite que mon code est sensée gérer, si je test avec deux nombres égaux :

```
>>> %Run 'chap 3.py'

Entrez le premier nombre : 94
Entrez le deuxieme nombre : 94
Nombre de bits utilise : 8
94 et 94 sont egaux
```

FIGURE 24 – Test avec a et b égaux

Conclusion

Cet exercice demandait de comparer deux nombres représentés en complément à 2 en se basant uniquement sur leurs bits. Pour y répondre, j'ai écrit un programme qui commence par calculer le nombre de bits nécessaires pour représenter les deux nombres puis qui compare leur bit de signe avant de comparer les bits restants un par un. Le programme prend en compte les cas suivants : deux nombres positifs, deux nombres négatifs, un positif et un négatif et deux nombres égaux.

Cet exercice m'a permis de mieux comprendre comment fonctionne le complément à 2.

J'ai compris que :

- le bit le plus à gauche, appelé bit de signe, détermine si un nombre est positif ou négatif : 0 pour positif, 1 pour négatif ;
- quand deux nombres ont des signes différents, le positif est forcément plus grand ;
- quand deux nombres ont le même signe, la comparaison des bits restants ne fonctionne pas de la même façon selon qu'ils sont positifs ou négatifs : pour les négatifs, une valeur binaire plus grande correspond en réalité à un nombre décimal plus petit.

2.3 Exercice 3.15 (Bonus)

Identifier les deux valeurs pour lesquelles le complément à deux ne change pas le bit de signe de la représentation d'un nombre.

Il est important de ne pas oublier d'expliquer et de justifier votre réponse.

Pour vous aider : relire l'encadré "Important" du chapitre.

En complément à 2, le bit de signe (le bit le plus à gauche) indique si un nombre est positif (0) ou négatif (1). L'opération de complément à 2 correspond à l'inversion de tous les bits suivie de l'addition de 1 dans une représentation sur n bits. Cette opération permet d'obtenir l'opposé d'un nombre.

On cherche ici les deux valeurs pour lesquelles le complément à 2 ne modifie pas le bit de signe, c'est-à-dire les valeurs qui restent inchangées après cette transformation.

1. Cas de la valeur 0

La valeur 0 est représentée sur n bits par une suite de bits tous égaux à 0 :

$$000\dots 0$$

En appliquant le complément à 2 à la valeur 0, on suit deux étapes successives.

- **Inversion des bits** : chaque bit 0 devient 1 et chaque bit 1 devient 0. Ainsi, sur n bits, la représentation $000\dots 0$ devient $111\dots 1$.
- **Addition de 1** : on ajoute 1 à la valeur obtenue :

$$111\dots 1 + 1$$

Cette addition dépasse la capacité de représentation sur n bits. En effet, sur un système à n bits, seuls les n bits de droite sont conservés. Le bit supplémentaire (retenue) est donc ignoré.

Ainsi, on obtient :

$$000\dots 0$$

Ainsi, 0 reste inchangé après l'opération de complément à 2.

2. Cas de la valeur -2^{n-1}

La deuxième valeur particulière est la plus petite valeur représentable sur n bits :

$$-2^{n-1}$$

Sa représentation binaire est :

$$1000\dots 0$$

Par exemple, sur 4 bits, cette valeur est -8 et s'écrit :

$$1000$$

Appliquons le complément à 2 à la valeur 1000 sur 4 bits.

- **Inversion des bits** : chaque bit est inversé (0 devient 1 et 1 devient 0). Ainsi :

$$1000 \rightarrow 0111$$

- **Addition de 1** : on ajoute 1 au résultat obtenu :

$$0111 + 1 = 1000$$

On retrouve exactement la même représentation binaire qu'au départ. Cela signifie que la valeur ne change pas après l'application du complément à 2, et que le bit de signe reste inchangé.

On retrouve donc la même représentation binaire, ce qui signifie que la valeur reste inchangée et que le bit de signe (1) ne change pas.

Ainsi, les deux seules valeurs pour lesquelles le complément à 2 ne modifie pas le bit de signe sont :

$$0 \quad \text{et} \quad -2^{n-1}$$

Conclusion

Cet exercice demandait d'identifier les deux valeurs pour lesquelles le complément à 2 ne change pas le bit de signe. En s'appuyant sur l'encadré "Important" du chapitre, qui rappelle que toutes les opérations en complément à 2 se font sur un nombre défini de bits, on peut montrer que ces deux valeurs sont 0 et la plus petite valeur représentable sur n bits, soit -2^{n-1} .

J'ai compris que :

- le complément à 2 d'un nombre consiste à inverser tous ses bits puis à ajouter 1 ;
- pour 0, inverser tous les bits donne une suite de 1, et ajouter 1 redonne une suite de 0 : le bit de signe reste donc 0 ;
- pour la valeur -2^{n-1} , représentée par 1000...0, l'inversion donne 0111...1 et l'ajout de 1 redonne 1000...0 : le bit de signe reste donc 1 ;
- ces deux cas sont les seuls où la valeur reste inchangée après l'opération et cette propriété est indépendante du nombre de bits utilisé.

2.4 Conclusion du chapitre 3

Ce chapitre portait sur les opérations en binaire et la représentation des nombres signés.

La deuxième partie, sur la représentation des nombres signés, selon moi, était la plus dense. Le complément à 2 est la notion qui m'a demandé le plus d'attention, dans le sens où il fallait comprendre pourquoi le bit de poids fort joue le rôle d'un bit de signe, pourquoi on peut additionner deux nombres signés sans se soucier de leur signe, ou encore pourquoi 0 et -1 sont les deux seules valeurs dont le complément à 2 ne change pas le bit de signe.

Pour résumer les notions vues dans ce chapitre :

- un décalage à gauche revient à multiplier un nombre par sa base, un décalage à droite revient à le diviser alors qu'en binaire, cela correspond à des multiplications et divisions par 2 ;
- la représentation la plus courante pour les entiers signés dans un ordinateur est le complément à 2 : on part de la représentation binaire de la valeur absolue, on inverse tous les bits, puis on ajoute 1 ;
- en complément à 2, le bit le plus à gauche joue le rôle de bit de signe : 0 pour un nombre positif, 1 pour un nombre négatif ;
- toutes les opérations en complément à 2 s'effectuent sur un nombre défini de bits.

Dans l'ensemble, c'est un chapitre qui m'a semblé plus exigeant que le précédent mais qui reste abordable.

3 Sources du chapitre 3

- Kislin-Duval, P. (2022). *Architecture des Machines et des Ordinateurs : Support de Cours* (Version 1.2). Institut d'Enseignement à Distance, Licence Informatique.
- Wikipédia. (2023). *Système négabinaire*. https://fr.wikipedia.org/wiki/Syst%C3%A8me_n%C3%A9gabinaire
- Cours de NSI (Terminale) — notions sur le complément à 2 et les comparaisons en binaire.

Chapitre 4 — Représentation des Nombres Flottants et des Caractères

Une Première Série d'Exercices

3.1 Exercice 4.3 (A Rendre)

Une légende dit qu'en guise de récompense, l'inventeur (indien) du jeu d'échecs demande un grain de blé sur la première case de l'échiquier, deux grains sur la seconde, quatre sur la troisième et ainsi de suite jusqu'à la soixante-quatrième.

1. Approximer en base 10 le nombre de grains à poser sur la dernière case.
2. En supposant qu'il faut dix grains de blé pour faire un gramme, approximer le poids que cela représente. (Convertir le résultat en tonnes.)

Réponse :

Les grains de riz évoluent de cette manière :

- 1 grain sur la première case,
- 2 grains sur la deuxième case,
- 4 grains sur la troisième case,
- et ainsi de suite jusqu'à la 64^e case (puisque sur un jeu d'échecs on a 64 cases).

On remarque que chaque case contient le double de la case précédente. On reconnaît ici une suite géométrique de raison 2, où le premier terme est 1.

On cherche le nombre de grains sur la dernière case.

Pour une suite géométrique, le n -ième terme u_n est donné par la formule :

$$u_n = u_1 \times r^{n-1}$$

Dans notre cas :

- $u_1 = 1$ (le premier terme),
- $r = 2$ (la raison),
- $n = 64$ (la dernière case).

Donc le nombre de grains sur la dernière case est :

$$u_{64} = 1 \times 2^{63} = 2^{63}$$

On peut l'approximer en base 10. On sait que :

$$2^{10} \approx 10^3$$

Donc :

$$2^{60} = (2^{10})^6 \approx (10^3)^6 = 10^{18}$$

Puis :

$$2^{63} = 2^3 \times 2^{60} = 8 \times 10^{18} \approx 8 \cdot 10^{18} \text{ grains.}$$

Donc sur la dernière case, il y a environ 8 milliards de milliards de grains.

Il faut 10 grains pour faire 1 gramme.

- Poids en grammes :

$$\text{poids (g)} = \frac{2^{63}}{10} \approx 8 \times 10^{17} \text{ g}$$

— Convertir en kilogrammes : $1 \text{ kg} = 1000 \text{ g}$

$$\text{poids (kg)} = \frac{8 \times 10^{17}}{10^3} = 8 \times 10^{14} \text{ kg}$$

— Convertir en tonnes : $1 \text{ tonne} = 1000 \text{ kg}$

$$\text{poids (tonnes)} = \frac{8 \times 10^{14}}{10^3} = 8 \times 10^{11} \text{ tonnes}$$

Donc :

- Le nombre de grains sur la dernière case est environ 8×10^{18} .
- Le poids correspondant est environ 8×10^{11} tonnes.

Conclusion

Cet exercice demandait d'approximer le nombre de grains sur la dernière case d'un échiquier, en sachant que chaque case contient le double de la précédente, puis d'en déduire le poids correspondant en tonnes. En reconnaissant une suite géométrique de raison 2, on obtient 2^{63} grains sur la dernière case, soit environ 8×10^{18} grains, ce qui correspond à environ 8×10^{11} tonnes.

Cet exercice était plutôt accessible car il repose sur des notions de mathématiques que j'ai vu au lycée notamment les suites géométriques. Reconnaître que le nombre de grains double à chaque case permet d'appliquer directement la formule $u_n = u_1 \times r^{n-1}$ et d'obtenir 2^{63} . Ce qui m'a davantage surpris, c'est le résultat final puisqu'on part d'un seul grain et on arrive à un nombre de l'ordre de 10^{18} .

3.2 Exercice 4.6 (Bonus)

Si nous nous référons au programme vu au chapitre précédent et rappelé ici :

1. Évaluer l'ordre de grandeur d'un nombre de tours de boucles qu'il effectue. En considérant ici que chaque tour de boucle fait trois opérations : (comparer avec 0, ajouter 1, recommencer la boucle) et que chaque opération prend un tic d'horloge (la fréquence de votre processeur indique le nombre de tics d'horloge par seconde).
2. Évaluer l'ordre de grandeur du temps nécessaire pour exécuter le programme. Placer ensuite ce code du programme (par copier-coller) dans un fichier nommé `foo.c`, le compiler avec la ligne de commande `gcc foo.c`, mesurer son temps d'exécution avec la ligne de commande `time a.out`.
3. Comparer le temps mesuré avec l'estimation "évaluée" de (2). Donnez quelques éléments de justification (c'est le plus important).

Pour vous aider : Quelques recherches à effectuer sur internet vous donneront quelques éléments de réponses...

Rappel du programme :

```
1 int main() {  
2     int t = 1;  
3     while (t != 0) {  
4         t = t + 1;  
5     }  
6     return 0;  
7 }
```

Réponse :

1 : Évaluer l'ordre de grandeur du nombre de tours de boucle

On initialise `t` à 1. La boucle continue tant que `t != 0`. À chaque tour, on ajoute 1 à `t`.

Ainsi :

- `t` est un entier de type `int`, souvent représenté sur 32 bits.
- La valeur maximale pour un entier 32 bits est $2^{31} - 1 = 2\,147\,483\,647$.
- La boucle effectue donc environ 2×10^9 tours avant de s'arrêter.

Nombre d'opérations par tour :

1. Comparer `t != 0`
2. Ajouter 1 à `t`
3. Recommencer la boucle

Donc pour 2×10^9 tours, le programme réalise environ 6×10^9 opérations au total.

2 : Estimation du temps

Chaque opération prend approximativement un tic d'horloge. Le processeur utilisé fonctionne à **3.6 GHz**, soit 3.6×10^9 tics par seconde.

Processeur 12th Gen Intel(R) Core(TM) i7-12700KF (3.60 GHz)

FIGURE 25 – Fréquence du processeur utilisée pour l'estimation (3.6 GHz).

Le temps d'exécution est donc :

$$t = \frac{6 \times 10^9}{3.6 \times 10^9} \approx 1,67 \text{ secondes}$$

Ainsi, le programme met environ **1,7 seconde** à s'exécuter.

3 : Comparer avec le temps mesuré

On vérifie ce résultat :

real 0m1.679s

user 0m1.679s

sys 0m0.000s

Pour répondre à cette question, j'ai utilisé plusieurs sources :

- <https://apprendre-la-programmation.net/estimer-le-temps-developpement-efficacement/> (même si elle ne parle pas de code "pur", je trouve que les exemples utilisés illustrent bien le problème)
- <https://interstices.info/sciences-de-linformation-la-ou-le-temps-sous-tend-tant-et-tant/> (pour tout ce qui est complexité/nombre d'opérations réelles, temps d'accès, etc.)
- <https://cahier-de-prepa.fr/psi-michelet/download?id=239> (pour la complexité également)

Le temps mesuré pour exécuter le programme est légèrement différent de l'estimation théorique pour plusieurs raisons. Tout d'abord, l'estimation ne prend en compte que le nombre d'opérations de base et la fréquence du processeur, en supposant que chaque opération dure exactement un tic d'horloge. En vérité, le processeur réalise également d'autres opérations telles que la gestion du cache, des interruptions et des processus de fond, ce qui peut avoir un impact sur le temps global. Par ailleurs, notre programme comporte une boucle qui augmente de manière linéaire avec la variable t , et la complexité du programme est directement liée à la dimension du type entier utilisé (en l'occurrence 32 bits). Si nous avions opté pour un nombre plus élevé, la durée aurait été considérablement plus longue.

On peut aussi comparer cette situation à l'estimation du temps dans un projet réel. Une société qui missionne un développeur a besoin de connaître le coût d'une tâche pour faire un devis, mais le temps estimé ne reflète jamais tout le travail réel : coder, réfléchir, organiser, se tromper, recommencer ou se former sont des éléments souvent sous-estimés ou oubliés. De plus, l'estimation varie selon la personne qui réalise la tâche : ce qui prend une journée à un développeur peut en prendre deux à un autre, ou moins à un autre encore. Ainsi, l'estimation n'est jamais parfaitement objective.

Finalement, certains logiciels ou systèmes d'exploitation peuvent occasionner de légères variations dans l'évaluation du temps réel, et la façon dont l'ordinateur accède à la mémoire ou aux registres peut également affecter le temps écoulé. Ainsi, bien que l'évaluation fournisse une approximation correcte, la durée réelle peut varier légèrement en raison de ces éléments matériels, logiciels et humains, ce qui est à la fois normal et prévisible.

Conclusion

Cet exercice demandait d'estimer le nombre de tours de boucle du programme, d'en déduire un temps d'exécution théorique, puis de le comparer au temps mesuré en pratique. En sachant qu'un `int` est représenté sur 32 bits, la boucle effectue environ 2×10^9 tours, soit 6×10^9 opérations au total. Avec un processeur à 3,6 GHz, le temps théorique estimé est d'environ 1,7 seconde ce que le test réel a confirmé avec un temps mesuré de 1,679 seconde.

J'ai compris que :

- un `int` est représenté sur 32 bits, ce qui fixe une valeur maximale de $2^{31} - 1$ et explique pourquoi la boucle finit par s'arrêter quand `t` déborde et repasse à 0 ;

- la fréquence du processeur, exprimée en GHz, représente le nombre d'opérations par seconde ;
- le temps mesuré est légèrement différent du temps théorique parce que le processeur effectue en réalité d'autres tâches en parallèle (gestion du cache, interruptions...) qui ne sont pas prises en compte dans l'estimation.

Une Seconde Série d'Exercices

Vous devez envoyer, pour valider ce quatrième chapitre, les exercices **4.3 (A Rendre)** et **4.13 (A Rendre)**. Pour celles et ceux qui souhaitent un petit plus : le **4.6** est pour un Bonus, l'exercice supplémentaire **4.17** ajoutera un second Bonus.

3.3 Exercice 4.13 (A Rendre)

Écrire un programme dans le langage de votre choix pour multiplier deux nombres représentés en base 10 et en virgule flottante entrés par l'utilisateur.

Penser à normaliser le résultat de façon à n'avoir qu'un seul chiffre à gauche de la virgule dans le résultat.

Réponse :

```
1  """
2  /*****
3  * Nom      : Chap_4_ex_4.13.py
4  * Role     : Multiplie deux nombres flottants et affiche le resultat
5  *          : normalise en notation scientifique
6  * Auteur   : Emma BARETS
7  * Version  : Version finale du 02/02/2026
8  * Licence  : Realise dans le cadre du cours Architecture des ordinateurs
9  * Dependances : Aucun
10 * Usage    : Executer le script et saisir deux nombres flottants
11 *****/
12 """
13
14 try:
15     # Si une erreur se produit dans ce bloc, Python passera dans le except
16
17     # Demande a l'utilisateur de saisir un premier nombre flottant
18     a = float(input("Entrez le premier nombre : "))
19
20     # Demande a l'utilisateur de saisir un deuxieme nombre flottant
21     b = float(input("Entrez le deuxieme nombre : "))
22
23     # Multiplication des deux nombres saisis
24     resultat = a * b
25
26     # Initialisation de l'exposant a 0
27     # Il servira a memoriser le nombre de decalages effectues
28     expo = 0
29
30     # Verification que le resultat n'est pas nul
31     if resultat != 0:
32
33         # Tant que la valeur absolue du resultat est superieure ou egale a 10
34         while abs(resultat) >= 10:
35
36             # On divise le resultat par 10
37             resultat = resultat / 10
38
```

```

39         # On augmente l'exposant de 1
40         expo = expo + 1
41
42         # Tant que la valeur absolue du resultat est inferieure a 1
43         while abs(resultat) < 1:
44
45             # On multiplie le resultat par 10
46             resultat = resultat * 10
47
48             # On diminue l'exposant de 1
49             expo = expo - 1
50
51         # Affichage de la mantisse normalisee
52         print("Resultat normalise :", resultat)
53
54         # Affichage de l'exposant calcule
55         print("Exposant :", expo)
56
57         # Affichage du resultat complet sous forme scientifique
58         print("Resultat en notation scientifique :", str(resultat) + " x 10^" + str(expo)
59               )
60
61     # Gestion de l'erreur si l'utilisateur n'entre pas un nombre valide
62     except ValueError:
63
64         # Message d'erreur affiche a l'utilisateur
65         print("Erreur : veuillez entrer des nombres valides.")

```

Pour cet exercice j'ai choisi Python car c'est le langage avec lequel je suis le plus à l'aise pour le moment. Il y a cours de "programmation impérative" en L1 en parallèle qui me permettra à terme de travailler en C, mais j'en suis encore au tout début.

```

1 try:

```

Le bloc `try` permet de surveiller les instructions susceptibles de provoquer une erreur. Ici, il sert principalement à vérifier que les valeurs saisies par l'utilisateur peuvent bien être converties en nombres flottants.

```

1 a = float(input("Entrez le premier nombre : "))
2 b = float(input("Entrez le deuxieme nombre : "))

```

On demande à l'utilisateur de saisir deux nombres. `input()` retourne toujours une chaîne de caractères, donc on utilise `float()` pour convertir directement la saisie en nombre à virgule flottante.

```

1 resultat = a * b

```

On multiplie les deux nombres et on stocke le résultat. À ce stade, le résultat n'est pas encore normalisé ce qui veut dire qu'il peut avoir plusieurs chiffres à gauche de la virgule ou être très petit.

```
1 expo = 0
```

On initialise un exposant à 0. Cet exposant va nous permettre de suivre combien de fois on a multiplié ou divisé le résultat par 10 pour le normaliser.

```
1 if resultat != 0:
2     while abs(resultat) >= 10:
3         resultat = resultat / 10
4         expo = expo + 1
```

Si le résultat est trop grand (supérieur ou égal à 10 en valeur absolue), on le divise par 10 à chaque tour de boucle et on incrémente l'exposant. On utilise `abs()` pour gérer correctement les nombres négatifs car la valeur absolue permet de comparer sans se soucier du signe.

```
1     while abs(resultat) < 1:
2         resultat = resultat * 10
3         expo = expo - 1
```

Si au contraire le résultat est trop petit (inférieur à 1 en valeur absolue), on le multiplie par 10 à chaque tour et on décrémente l'exposant. Ces deux boucles garantissent qu'on obtient toujours exactement un chiffre à gauche de la virgule, ce qui correspond à la normalisation demandée dans l'énoncé.

```
1 print("Resultat normalise :", resultat)
2 print("Exposant :", expo)
3 print("Resultat en notation scientifique :", str(resultat) + " x 10^" + str(expo))
```

On affiche le résultat sous trois formes : la mantisse, l'exposant seul, puis la notation scientifique complète sous la forme $m \times 10^n$. On utilise `str()` pour convertir les nombres en chaînes de caractères afin de pouvoir les concaténer dans le `print`.

```
1 except ValueError:
```

Le bloc `except ValueError` est exécuté lorsqu'une erreur de conversion est détectée. Une erreur `ValueError` apparaît par exemple si l'utilisateur saisit du texte comme "bonjour" au lieu d'un nombre.

```
1 print("Erreur : veuillez entrer des nombres valides.")
```

Si une erreur est détectée, le programme affiche un message explicite à l'utilisateur au lieu de s'arrêter brutalement avec un message d'erreur Python.

Test pour vérifier que tout fonctionne bien (code en annexe) :



Pas de soucis pour un cas "normal".

On teste pour deux très petits nombres $a = 0.001$ et $b = 0.002$. On devrait obtenir 2.0×10^{-6} .

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparati
on_envoi_chap_2_a_5\Annexe> python3 Chap_4_ex_4.13.py
Entrez le premier nombre : 0.001
Entrez le deuxième nombre : 0.002
Résultat normalisé : 2.0
Exposant : -6
Résultat en notation scientifique : 2.0 x 10^-6
```

FIGURE 26 – Test avec deux très petits nombres.

Le résultat est correct.

Ensuite, on teste un nombre négatif $a = -3.5$ et un nombre positif $b = 2.0$. On devrait obtenir -7.0×10^0 .

```
on_envoi_chap_2_a_5\Annexe> python3 Chap_4_ex_4.13.py
Entrez le premier nombre : -3.5
Entrez le deuxième nombre : 2
Résultat normalisé : -7.0
Exposant : 0
Résultat en notation scientifique : -7.0 x 10^0
```

FIGURE 27 – Test avec un nombre négatif et un nombre positif.

On obtient bien -7.0×10^0 .

On teste ensuite avec $a = 0.0$ et $b = 5.0$.

```
on_envoi_chap_2_a_5\Annexe> python3 Chap_4_ex_4.13.py
Entrez le premier nombre : 0
Entrez le deuxième nombre : 5
Résultat normalisé : 0.0
Exposant : 0
Résultat en notation scientifique : 0.0 x 10^0
```

FIGURE 28 – Test avec zéro.

Le résultat est correct.

Dernier test avec la gestion d'erreur, pour éviter que l'utilisateur rentre quelque chose d'invale :

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparat
ion_envoi_chap_2_a_5\Annexe> python3 Chap_4_ex_4.13.py
Entrez le premier nombre : bonjour
Erreur : veuillez entrer des nombres valides.
```

FIGURE 29 – Test avec bonjour.

On nous renvoie bien : "Erreur : veuillez entrer des nombres valides."

Conclusion

Cet exercice demandait d'écrire un programme qui multiplie deux nombres en virgule flottante et normalise le résultat pour n'avoir qu'un seul chiffre à gauche de la virgule. Le programme demande deux nombres à l'utilisateur, les multiplie, puis ajuste le résultat en le divisant ou le multipliant par 10 autant de fois que nécessaire, tout en maintenant un exposant pour garder la trace des ajustements effectués. Le résultat est ensuite affiché sous forme de notation scientifique.

La partie qui m'a demandé le plus de réflexion, c'est la normalisation. Au départ je savais ce que je voulais obtenir, un seul chiffre à gauche de la virgule, mais je ne savais pas trop comment l'implémenter. J'ai raisonné de cette façon : pour qu'un nombre ait exactement un chiffre à gauche de la virgule, il doit être compris entre 1 et 10. Donc si le résultat est trop grand, il faut le diviser par 10, et s'il est trop petit, il faut le multiplier par

10. La question qui s'est posée ensuite c'est : combien de fois ? Comme on ne peut pas le savoir à l'avance, un `if` ne suffit pas donc il faut une boucle `while` qui continue tant que le résultat n'est pas dans le bon intervalle. Et à chaque fois qu'on divise ou multiplie par 10, il faut garder trace du nombre d'opérations effectuées, d'où l'idée de l'exposant qu'on incrémente ou décrémenté à chaque tour.

J'ai également compris que :

- si le résultat est supérieur ou égal à 10, on le divise par 10 et on incrémente l'exposant jusqu'à ce qu'il soit dans le bon intervalle ;
- si le résultat est inférieur à 1, on le multiplie par 10 et on décrémenté l'exposant ;
- il faut utiliser `abs()` pour comparer les valeurs en ignorant le signe, sinon un nombre négatif comme -7.0 serait mal géré par les boucles ;
- le cas où le résultat vaut 0 doit être traité séparément pour éviter une boucle infinie.

3.4 Exercice 4.17 (Bonus)

Écrire dans le langage de votre choix un programme qui calcule la fonction exponentielle selon la formule de Taylor. Constaté l'erreur d'approximation produite.

Rappel de la formule :

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots + \frac{x^n}{n!}$$

Explication du code :

```
1 x = float(input("Entrez la valeur de x : "))
2 n = int(input("Entrez le nombre de termes pour l'approximation (plus grand = plus
    precis) : "))
```

On demande deux valeurs à l'utilisateur : x , la valeur pour laquelle on veut calculer e^x , et n , le nombre de termes de la série de Taylor. On utilise `float()` pour x car il peut avoir une partie décimale, et `int()` pour n car un nombre de termes est forcément un entier, on ne peut pas avoir 7,5 termes.

```
1 exp = 1.0
```

On initialise la somme à 1.0 et pas à 0. Pourquoi ? Parce qu'en regardant la formule de Taylor, le premier terme est toujours 1 (c'est $\frac{x^0}{0!} = 1$). On part donc directement avec ce premier terme et la boucle s'occupera des suivants.

```
1 fact = 1
2 puissance = 1
```

On initialise deux variables en dehors de la boucle. On aurait pu recalculer la factorielle et la puissance depuis zéro à chaque tour, mais ce serait inefficace parce que on aurait calculer x^3 , on aurait recalculé x^1 , x^2 puis x^3 à chaque fois. Ici on garde en mémoire le résultat précédent et on le met à jour à chaque tour c'est plus rapide et plus simple.

```
1 for i in range(1, n+1):
2     puissance = puissance * x
3     fact = fact * i
4     terme = puissance / fact
5     exp = terme + exp
```

À chaque tour de boucle, on calcule le terme suivant de la série de Taylor. À chaque tour, `puissance` est multiplié par x (ce qui donne x^i) et `fact` est multiplié par i (ce qui donne $i!$). On divise ensuite l'un par l'autre pour obtenir le terme $\frac{x^i}{i!}$, qu'on ajoute à la somme.

```
1 print("Approximation de e^", x, "avec", n, "termes : ", exp)
```

On affiche le résultat final. On reprend x et n dans le message pour que l'affichage soit clair.

J'ai également ajouté une gestion d'erreur avec `try` et `except` afin d'éviter que le programme s'arrête si l'utilisateur saisit autre chose qu'un nombre. Dans ce cas, un message d'erreur clair est affiché.

Code finale :

Voici le code final :

```
1  """
2  /*****
3  * Nom      : Chap_4_ex_4.17.py
4  * Role     : Multiplie deux nombres flottants et affiche le resultat
5  *           normalise en notation scientifique
6  * Auteur   : Emma BARETS
7  * Version  : Version final du 03/02/2026
8  * Licence  : Realise dans le cadre du Architecture des ordinateurs
9  * Dependances : Aucun
10 * Usage     : Executer le script et saisir deux nombres flottants
11 *****/
12 """
13
14 try:
15     # Entrer la valeur de x
16     x = float(input("Entrez la valeur de x : "))
17
18     # Entrer le nombre de termes n
19     n = int(input("Entrez le nombre de termes pour l'approximation : "))
20
21     # Verification que n est positif
22     if n < 0:
23         print("Erreur : le nombre de termes doit etre positif.")
24     else:
25
26         # Initialisation de la somme avec le premier terme
27         exp = 1.0
28
29         # Variables utilisees pour calculer les termes successifs
30         fact = 1
31         puissance = 1
32
33         # Calcul de la serie de Taylor
34         for i in range(1, n + 1):
35             puissance = puissance * x
36             fact = fact * i
37             terme = puissance / fact
38             exp = exp + terme
39
40         # Affichage du resultat
41         print("Approximation de e^", x, "avec", n, "termes :", exp)
42
43 except ValueError:
44     print("Erreur : veuillez entrer des valeurs numeriques valides.")
```

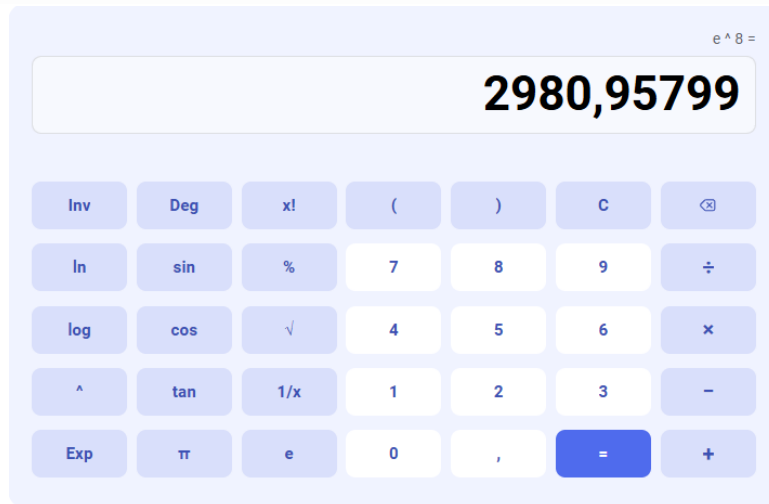
Test pour verifier que le programme fonctionne :

```
>>> %Run 'chap 4 17.py'
```

```
Entrez la valeur de x : 8
```

```
Entrez le nombre de termes pour l'approximation (plus grand = plus précis) : 6
```

```
Approximation de  $e^8$  avec 6 termes : 934.1555555555556
```



Le programme retourne la valeur 934.1555555555556. Ce résultat reste assez éloigné de la valeur réelle de e^8 , qui vaut environ 2980.958. Cela s'explique par le fait que seulement 6 termes de la série de Taylor sont utilisés. L'approximation est donc encore peu précise.

Plus x est grand et/ou n est petit, plus l'erreur sera enorme (par exemple avec $x = 656$ et $n = 4$) :

```
>>> %Run 'chap 4 17.py'
```

```
Entrez la valeur de x : 656
```

```
Entrez le nombre de termes pour l'approximation (plus grand = plus précis) : 4
```

```
Approximation de  $e^{656}$  avec 4 termes : 7763477265.0
```



Lorsque la valeur de x augmente les termes de la serie de Taylor devienne plus important ce qui demande une precision et donc davantage de chiffre apres la virgule.

Je test avec un plus grand nombre :

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparation_envoie_chap_2_a_5\Annexe> python3 Chap_4_ex_4.17.py
Entrez la valeur de x : 8
Entrez le nombre de termes pour l'approximation : 50
Approximation de e^ 8.0 avec 50 termes : 2980.957987041728
```

FIGURE 30 – Test un plus grand nombre.

Ce résultat est quasiment identique à la valeur réelle de $e^8 \approx 2980.957987$. Cela montre que lorsque le nombre de termes augmente, l'approximation devient de plus en plus précise et converge vers la valeur exacte.

Dernier test, cette fois pour vérifier que le programme ne bug pas si on rentre autre chose que ce qui est demandé :

```
PS C:\Users\emma5\OneDrive\Documents\L1-informatique\Semestre-2\UE-informatique-2\Architecture-des-ordinateurs\Préparation_envoie_chap_2_a_5\Annexe> python3 Chap_4_ex_4.17.py
Entrez la valeur de x : Bonjour
Erreur : veuillez entrer des valeurs numeriques valides.
```

FIGURE 31 – Test de gestion d'erreur.

On a bien : "Erreur : veuillez entrer des valeurs numeriques valides."

Conclusion

Cet exercice demandait d'implémenter la fonction exponentielle à partir de la formule de Taylor et de constater l'erreur d'approximation produite. Le programme demande à l'utilisateur une valeur x et un nombre de termes n , puis calcule la somme des termes de la série de Taylor de manière progressive. Plus n est grand, plus l'approximation est précise et inversement, plus x est grand avec un n petit, plus l'erreur sera importante.

Cet exercice m'a fait découvrir la formule de Taylor, que je ne connaissais pas du tout. Au départ j'ai eu un peu peur, les maths ne sont pas forcément mon point fort, mais une fois la formule comprise, ça a été.

J'ai compris que :

- la série de Taylor permet d'approcher une fonction complexe comme e^x en additionnant des termes de plus en plus petits et que plus on en ajoute, plus on se rapproche de la vraie valeur ;
- l'erreur d'approximation n'est pas un bug. Plus x est grand, plus les termes de la série sont importants et plus il en faut pour obtenir une bonne précision.

3.5 Conclusion du chapitre 4

Ce chapitre portait sur la représentation des nombres flottants et des caractères. La première partie sur les ordres de grandeur entre les bases 2 et 10 m'a montré qu'on peut faire des estimations rapides en exploitant le fait que $2^{10} \approx 10^3$ (ex : exercice 4.6 pour estimer le temps d'exécution d'un programme). La partie sur la virgule flottante m'a permis de comprendre comment un ordinateur représente des nombres qui ne sont pas entiers et surtout pourquoi cette représentation est toujours une approximation. Avec un nombre fixe de bits pour la mantisse, certains nombres comme 0.3 ne peuvent tout simplement pas être représentés exactement en base 2. La norme IEEE 754 fixe comment ces bits sont répartis entre le signe, l'exposant et la mantisse et gère des cas particuliers comme l'infini. L'exercice 4.17 sur la formule de Taylor était celui où j'ai eu le plus de difficultés, les maths m'ont fait un peu peur au départ mais j'ai bien compris pourquoi l'erreur augmente quand x est grand et n petit.

Pour résumer les notions principales vues dans ce chapitre :

- $2^{10} \approx 10^3$ est une approximation utilisée pour estimer rapidement des ordres de grandeur en informatique ;
- un nombre en virgule flottante se décompose en un signe, une mantisse et un exposant — la valeur du nombre est $(-1)^s \times m \times 2^e$;
- la représentation en virgule flottante est toujours une approximation : il est impossible de représenter exactement certains nombres comme 0.3 en base 2 avec un nombre fini de bits ;
- la norme IEEE 754 est le standard utilisé par les ordinateurs modernes pour représenter les nombres flottants, sur 32 bits (simple précision) ou 64 bits (double précision).

4 Sources du chapitre 4

- Kislin-Duval, P. (2022). *Architecture des Machines et des Ordinateurs : Support de Cours* (Version 1.2). Institut d'Enseignement à Distance, Licence Informatique.
- OpenClassrooms. (2013). *Combien de bits pour un nombre décimal ?* <https://openclassrooms.com/forum/sujet/combien-de-bits-pour-un-nombre-decimal>
- Apprendre-la-programmation.net. (2020). *Estimer le temps de développement efficacement.* <https://apprendre-la-programmation.net/estimer-le-temps-developpement-efficacement/>
- Interstices.info. (2006). *Sciences de l'information : là où le temps sous-tend tant et tant.* <https://interstices.info/sciences-de-linformation-la-ou-le-temps-sous-tend-tant-et-tant/>
- Cahier de prépa PSI Michelet. (2021-2022). *Document sur la complexité algorithmique.* <https://cahier-de-prepa.fr/psi-michelet/download?id=239>
- Cours de NSI (Terminale) — suites géométriques, logarithmes, complexité, approximation de Taylor.

Chapitre 5 - Algèbre de Boole

Exercices

Pour ce cinquième chapitre, les exercices à compléter et à rendre dans le PDF sont : **5.10** (quatre équations au choix), **5.11** (deux tables au choix) et **5.9** pour un **Bonus** (Attention de bien respecter la consigne).

4.1 Exercice 5.9 (Bonus)

Imaginons que dans notre langage de tous les jours, nous utilisons l'algèbre de Boole. Que répondrait alors la sage-femme lorsque le nouveau père lui demande : « Alors, c'est un garçon ou une fille ? ».

Dans quel cas la réponse de l'infirmière serait-elle différente si le père lui demandait : « C'est un garçon ou exclusif une fille ? ».

Il est important d'utiliser ici l'ontologie (les mots-métiers) de la sage-femme et non celle du logisticien. La réponse doit être donnée sous forme de phrase naturelle, et non avec une expression logique. Il faut bien expliquer et développer vos réponses.

Réponse :

Quand le papa demande à la sage-femme : "Alors, c'est un garçon ou une fille ?", dans le langage de tous les jours, le mot "ou" veut dire "soit l'un, soit l'autre, mais pas les deux". La sage-femme sait donc instinctivement que le bébé ne peut être ni un garçon ni une fille en même temps et elle répondra simplement : "C'est un garçon" ou "C'est une fille".

En logique, ce "ou" correspond à ce qu'on appelle le "ou exclusif". Si on prenait le "ou" au sens logique inclusif, cela voudrait dire "un garçon ou une fille ou les deux", ce qui n'aurait pas de sens pour un seul bébé, sauf s'il s'agit de jumeaux de sexes différents. Dans ce cas précis, la réponse serait différente : on pourrait alors dire qu'il y a à la fois un garçon et une fille.

Ainsi, dans notre quotidien, on utilise sans s'en rendre compte le "ou exclusif", et ce n'est que dans des cas particuliers comme une naissance de jumeaux que le sens logique du mot "ou" (inclusif) pourrait modifier la réponse.

Cet exercice demandait de réfléchir à la différence entre le "ou" du langage courant et le "ou" en algèbre de Boole, en répondant sous forme de phrases naturelles sans expressions logiques. Dans la vie de tous les jours, on utilise instinctivement le "ou exclusif" sans s'en rendre compte. C'est seulement dans un cas particulier comme une naissance de jumeaux que le "ou inclusif" changerait la réponse.

Conclusion

Cet exercice était différent de ce qu'on fait habituellement dans ce chapitre, pas de code, pas d'expression logique, juste des mots et un raisonnement "naturel". Au départ j'ai eu un peu peur de ne pas assez développer, parce que la réponse me semblait évidente une fois qu'on y réfléchit. Mais je pense que justement c'est l'intérêt de l'exercice de montrer que l'algèbre de Boole on l'utilise en permanence dans le langage courant sans s'en rendre compte.

J'ai compris que :

- le "ou" du langage courant correspond en réalité au "ou exclusif" (XOR) de la logique booléenne : on dit "un garçon ou une fille" en sous-entendant que les deux sont impossibles en même temps ;
- le "ou inclusif" de la logique booléenne, lui, accepte que les deux conditions soient vraies simultanément, ce qui dans ce contexte ne ferait sens que pour des jumeaux de sexes différents.

4.2 Exercice 5.10 (À rendre)

Simplifier algébriquement les expressions algébriques de gauche, jusqu'à obtenir l'expression de droite.

Choisissez quatre équations ci-dessous selon le degré de difficulté souhaité. Gardez bien l'ordre alphabétique des équations choisies. Par exemple : (a, c, g, i).

b.

$$f = (a \cdot c + b \cdot \bar{c}) \cdot (\bar{a} + \bar{c}) \cdot b$$

Étapes de simplification :

$$\begin{aligned}(a \cdot c + b \cdot \bar{c}) \cdot (\bar{a} + \bar{c}) &= a \cdot c \cdot \bar{a} + a \cdot c \cdot \bar{c} + b \cdot \bar{c} \cdot \bar{a} + b \cdot \bar{c} \cdot \bar{c} && \text{(distribution)} \\ &= 0 + 0 + b \cdot \bar{c} \cdot \bar{a} + b \cdot \bar{c} && \text{(car } a \cdot \bar{a} = 0 \text{ et } c \cdot \bar{c} = 0) \\ &= b \cdot \bar{c} \cdot (\bar{a} + 1) && \text{(factorisation)} \\ &= b \cdot \bar{c} && \text{(puisque } \bar{a} + 1 = 1)\end{aligned}$$

c.

$$f = a \cdot \bar{b} \cdot \bar{c} + a \cdot b \cdot \bar{c} + a \cdot b \cdot c$$

Étapes de simplification :

$$\begin{aligned}f &= a \cdot (\bar{b} \cdot \bar{c} + b \cdot \bar{c} + b \cdot c) && \text{(factorisation par } a) \\ &= a \cdot ((\bar{b} + b) \cdot \bar{c} + b \cdot c) && \text{(factorisation par } \bar{c} \text{ dans les deux premiers termes)} \\ &= a \cdot (\bar{c} + b \cdot c) && \text{(car } \bar{b} + b = 1) \\ &= a \cdot (b + \bar{c}) && \text{(réécriture pour simplifier l'expression)}\end{aligned}$$

e.

$$f = (a + b) \cdot (\bar{a} + b)$$

Étapes de simplification :

$$\begin{aligned}f &= a \cdot \bar{a} + a \cdot b + b \cdot \bar{a} + b \cdot b && \text{(distribution)} \\ &= 0 + a \cdot b + b \cdot \bar{a} + b && \text{(car } a \cdot \bar{a} = 0 \text{ et } b \cdot b = b) \\ &= a \cdot b + b \cdot (\bar{a} + 1) && \text{(factorisation par } b) \\ &= a \cdot b + b && \text{(car } \bar{a} + 1 = 1) \\ &= b && \text{(absorption : } a \cdot b + b = b)\end{aligned}$$

f.

$$f = (x \cdot \bar{y} + z) \cdot (x + \bar{y}) \cdot z$$

Étapes de simplification :

$$\begin{aligned} (x \cdot \bar{y} + z) \cdot (x + \bar{y}) &= x \cdot \bar{y} \cdot x + x \cdot \bar{y} \cdot \bar{y} + z \cdot x + z \cdot \bar{y} && \text{(distribution)} \\ &= x \cdot \bar{y} + 0 + z \cdot x + z \cdot \bar{y} && \text{(car } x \cdot x = x, \bar{y} \cdot \bar{y} = \bar{y}) \\ &= x \cdot \bar{y} + z \cdot (x + \bar{y}) && \text{(factorisation par } z) \\ f &= (x \cdot \bar{y} + z \cdot (x + \bar{y})) \cdot z \\ &= (x + \bar{y}) \cdot z && \text{(simplification finale)} \end{aligned}$$

Conclusion

Cet exercice demandait de simplifier algébriquement quatre expressions booléennes en appliquant les règles de l'algèbre de Boole.

Cet exercice m'a semblé globalement accessible, la simplification algébrique, c'est quelque chose que je trouve pas très difficile. Ce qui m'a posé le plus de problèmes, ce ne sont pas les règles en elles-mêmes mais des erreurs bêtes d'inattention : oublier un terme, mal distribuer et me retrouver avec un résultat faux sans comprendre pourquoi au premier coup d'œil. Au final je m'en suis sorti, mais ça m'a rappelé qu'aller vite est souvent une mauvaise idée.

J'ai compris que :

- $a \cdot \bar{a} = F$ et $a + \bar{a} = V$ (complémentarité) ;
- $a \cdot V = a$ et $a + F = a$ (élément neutre) ;
- $V + a = V$ et $F \cdot a = F$ (élément absorbant) ;
- $a \cdot b + a = a$ et $a + a \cdot b = a$ (absorption) ;
- la distribution : $a \cdot (b + c) = a \cdot b + a \cdot c$ et $a + (b \cdot c) = (a + b) \cdot (a + c)$.

4.3 Exercice 5.11 (À rendre)

Utilisez les tables de Karnaugh suivantes pour obtenir la forme algébrique simplifiée de la fonction qu'elles décrivent.

Choisissez deux tables ci-dessous parmi les quatre. Comme pour l'exercice précédent, gardez bien l'ordre alphabétique des tables choisies. Par exemple : (b, d).

Table (a)

La table (a) est :

$cd \backslash ab$	FF	FV	VV	VF
FF	V	V	V	V
FV	V			V
VV	V			V
VF	V	V	V	V

On remarque que toutes les cases de la colonne FF et de la colonne VF contiennent V sur toutes les lignes. Cela signifie que la fonction est vraie pour toutes les valeurs de a et c , et ces V peuvent être couverts par le terme le plus simple qui inclut b et d . Ainsi, la fonction simplifiée s'écrit :

$$f_a(a, b, c, d) = b + d$$

Table (c)

La table (c) est :

$cd \backslash ab$	FF	FV	VV	VF
FF		V	V	
FV	V	V	V	
VV	V	V	V	V
VF		V	V	

Pour simplifier cette table, on identifie deux groupements de cases V :

- Les colonnes FV et VV contiennent des V sur toutes les lignes (8 cases). Ce grand rectangle couvre toutes les combinaisons où $b = V$, ce qui donne simplement le terme b ;
- Les deux cases de la colonne FF sur les lignes FV et VV (où $a = F$ et $d = V$) forment un rectangle de deux cases, ce qui donne le terme $\bar{a} \cdot d$.

En combinant ces deux termes, on obtient l'expression simplifiée :

$$f_c(a, b, c, d) = b + \bar{a} \cdot d$$

Conclusion

Cet exercice demandait d'utiliser les tables de Karnaugh pour obtenir la forme algébrique simplifiée de deux fonctions logiques. La fonction simplifiée est ensuite la réunion de tous ces termes.

Les tables de Karnaugh étaient une notion complètement nouvelle pour moi. La méthode m'a semblé mitigée dans le sens où d'un côté, une fois qu'on a compris le principe des groupements, c'est assez mécanique, automatiquement on cherche les rectangles les plus grands possibles et on en déduit les termes. De l'autre, identifier les bons groupements n'est pas toujours évident et il m'est arrivée de me tromper dans la lecture de la table, ce qui m'a donné un résultat faux notamment pour la table (c).

J'ai compris que :

- les groupements doivent toujours contenir un nombre de cases qui est une puissance de 2 (1, 2, 4, 8...);
- il faut maximiser la taille des groupements pour obtenir l'expression la plus simple possible, un grand rectangle élimine plus de variables qu'un petit;
- une variable est éliminée du terme quand elle change d'état à l'intérieur du groupement; seules les variables qui restent constantes apparaissent dans le terme final;
- une même case V peut appartenir à plusieurs groupements différents, ce qui est souvent nécessaire pour couvrir toutes les cases.

4.4 Conclusion du chapitre 5

Pour être honnête, ce chapitre est celui que j'ai le moins appréciée. Il y a beaucoup de maths et les maths ne sont pas vraiment mon point fort. Cela dit, le cours est bien expliquée et la progression est suffisamment claire pour que ce soit accessible même sans être à l'aise avec les formules.

Pour résumer les notions importantes vues dans ce chapitre :

- les trois fonctions de base sont **et** ($a \cdot b$), **ou** ($a + b$) et **non** ($\bar{a}$) ;
- le **ou** du langage courant correspond au **ou exclusif** (XOR) de la logique booléenne et non au **ou inclusif** ;
- pour simplifier algébriquement, on applique les règles de base : idempotence ($a \cdot a = a$), complémentarité ($a \cdot \bar{a} = F$), absorption ($a \cdot b + a = a$), dominance ($V + a = V$) et distribution ;
- les tables de Karnaugh permettent de simplifier une expression en regroupant les cases V en rectangles de 2^n cases et plus le rectangle est grand, plus le terme résultant est simple ;
- entre deux cases voisines dans une table de Karnaugh, une seule variable change de valeur.

5 Sources du chapitre 5

- Kislin-Duval, P. (2022). *Architecture des Machines et des Ordinateurs : Support de Cours* (Version 1.2). Institut d'Enseignement à Distance, Licence Informatique.